Supervised Spatially Spectrally Coherent Local Linear Embedding—Wasserstein Graph Convolutional Network for Hyperspectral Image Classification

Jairo Orozco, *Member, IEEE*, Sergio Manzanarez, *Member, IEEE*, and Vidya Manian, *Senior Member, IEEE*

***Abstract*—Hyperspectral image (HSI) classification faces challenges due to the high-dimensional and complex nature of the data. Graph convolutional network (GCN) is a widely used method for HSI classification. Conventional GCN analyzes fixed adjacency relations and ignores spatial spectral relationships. To overcome these issues, we propose a supervised spatially spectrally coherent local linear embedding integrated with Wasserstein distance-based GCN (S^{3}CLLE-WGCN) for HSI classification. This method combines the strengths of local linear embedding (LLE) for dimensionality reduction (DR) and WD for graph adjacency matrix construction, named Wasserstein GCN (WGCN) for effective feature extraction in non-Euclidean spaces. The supervised spatially spectrally coherent LLE (S^{3}CLLE) branch enhances the spectral–spatial manifold representation preserving local geometric variations and class boundaries while effectively reducing the dimensionality of the dataset, while the Earth mover's or WD better discriminates hyperspectral classes. The S^{3}CLLE-WGCN achieves state-of-the-art average accuracies: 98.44% (Indian Pines), 99.35% (Houston), 97.92% (Pavia), and 99.74% (Salinas Scene). These results demonstrate that S^{3}CLLE-WGCN outperforms the baselines in terms of various evaluation criteria.**

***Index Terms*—Adaptive neighborhood, dimensionality reduction (DR), graph convolutional network (GCN), hyperspectral image (HSI) classification, Laplacian matrix, local linear embedding (LLE), Wasserstein distance (WD).**

I. Introduction

HYPERSPECTRAL images (HSIs) have garnered attention due to the rich information they contain, and advancements in optical sensors have rendered these images more complex due to the substantial information embedded in the spatial, spectral, and temporal domains. The investigation of HSIs introduces distinctive challenges, predominantly attributable to their elevated information content. Machine learning (ML) methods have been widely used for HSI classification [1], [2], [3]. However, their effectiveness is hindered due to their limitations in discerning relationships among pixels. This limitation arises from their constrained capacity for local feature extraction. Consequently, overcoming this challenge necessitates the adoption of non-Euclidean space representations [4], which proves essential for mitigating the constraints associated with local feature extraction [5]. Recently, HSI classification has been addressed with greater success using deep learning methods.

Among the various deep learning techniques employed for HSI classification, CNNs have proven highly effective [6], [7]. The architecture of CNNs can vary in terms of structure, hidden layer count, and the inclusion of pooling layers [8]. Additionally, alternative models such as U-Net [9], [10], [11], which incorporate deconvolution and upsampling layers, have been explored. Some researchers have combined network structures like U-Net with recurrent networks [11], while others have employed 2-D and 3-D CNNs to enhance learning capacity, albeit with potential issues of saturation and reduced accuracy [11], [12].

Nevertheless, standard CNN models exhibit limitations when applied to HSIs. Their convolutional operations are confined to regular square regions, making them inadequate for capturing the geometric variations in different object regions within the HSI. Moreover, the fixed weights and size shapes of convolutional kernels lead to uniform weights during convolution across all image patches, contributing to the loss of class boundaries during feature abstraction and resulting in misclassifications [13]. Furthermore, common challenges associated with CNNs, such as overfitting, vanishing gradients, and convergence issues, persist when utilized for HSI classification [11]. Various ML algorithms have been developed to analyze diverse land cover types and classify them from remotely sensed HSIs [4]. These algorithms encompass pattern recognition methods, *k*-nearest neighbor, support vector machine (SVM), and sparse representation-based classifiers [13]. Nevertheless, distinguishing different land cover categories exclusively based on spectral information remains challenging. The spatial neighborhood of pixel spectra contains correlation information in images, prompting some researchers to explore spatial–spectral information. However, many approaches involve handcrafted spectral–spatial features, necessitating extensive computation and human intervention [13].

Received 30 July 2024; revised 9 December 2024 and 23 April 2025; accepted 27 August 2025. Date of publication 29 August 2025; date of current version 9 September 2025. This work was supported by the National Science Foundation under Grant 2334265. *(Corresponding author: Jairo Orozco.)*

The authors are with the Department of Electrical and Computer Engineering, University of Puerto Rico at Mayagüez, Mayagüez, PR 00682 USA (e-mail: jairo.orozco@upr.edu; sergiodavid.manzanarez@upr.edu; vidya.manian@upr.edu).

Digital Object Identifier 10.1109/TGRS.2025.3604376

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

In addressing these challenges, some researchers have sought alternative methods, including leveraging deep learning combined with graph-based techniques. Graph-based deep learning has proven effective in the processing and analysis of HSIs. This is especially applicable because an image can be effectively represented as a graph, where connections between pixels are established. For example, in images with an Euclidean domain, a graph representation can be formed where each pixel is a node, and edges represent connections between adjacent nodes [14], [15]. Numerous non-Euclidean data structures can be aptly described as graphs. While CNNs effectively capture hidden patterns in Euclidean data [14], they encounter limitations when applied to non-Euclidean spaces. A promising solution to this issue is the use of graph convolutional network (GCN). GCNs operate on graphs, allowing them to aggregate and transform feature information from the neighbors of each node in the network. The learned hidden layers encode node features and local graph structures, enabling GCNs to effectively leverage image features and preserve class boundaries [13].

GCNs are an invaluable approach for image processing tasks in nonregular domains, particularly when the inherent complexity of the data surpasses the capabilities of a conventional CNN. In deep learning methodologies, several techniques, including CNNs, tend to disregard location information during the high-level feature extraction stage, a critical aspect in tasks such as image semantic segmentation [16]. Similarly, synthetic aperture radar (SAR) imagery, characterized by oversegmentation, presents a non-Euclidean structure, making it well-suited for processing through graph-based deep learning methods [17].

Recently, interpretable graph neural networks with visual attention modules to focus on important regions have been developed for the detection of maritime objects from images [18]. Further improvements to object detection in remote sensing images by incorporating bilevel routing attention module are presented in [19]. Multiview graph representation with spectral–spatial features from extended morphological profile and semi-supervised GCN utilizing both labeled and unlabeled samples is proposed in [20] for HSI classification. An active learning method called prototype learning and GCN leveraging unlabeled samples is presented in [21] for enhancing HSI classification. A graph-structured convolution to extract discriminative features with cross attention mechanism for HSI classification with limited training samples is presented in [22].

However, applying GCN often entails a substantial computational cost due to the matrix representation of graphs and the requisite mathematical operations involved in graph convolution. To mitigate this challenge, Ma et al. [23] introduced a novel approach by constructing an adjacency matrix using pixels within localized patches containing rich spatial information. This methodology, combined with GCN and graph pooling, yields an effective classification map. To address computational cost concerns, Hong et al. [24] proposed MiniGCN, a method that facilitates the training of large-scale GCNs with small batches. MiniGCN employs an adjacency matrix constructed through k-nearest neighbors, incorporating CNN with GCN through element-wise addition, multiplication, and concatenation to enhance the classification performance.

Furthermore, researchers have explored the concept of superpixels to reduce computational expenses and glean information on ground objects in HSI. Jia et al. [25] proposed a superpixel-based method where each superpixel represents an internal graph, and connections among superpixels form an external graph, capturing both local and global characteristics of ground objects. Wan et al. [13] adopted a similar approach, partitioning HSIs into superpixels based on spectral and spatial similarities. This method trains a GCN with fewer nodes (superpixels) than the original image, generating an adjacency matrix where each node represents an embedding of a superpixel.

HSIs are characterized by hundreds of bands, and among them, certain bands exhibit redundancy and high correlation. The computational burden of processing the entirety of these bands is often substantial, as noted by Islam et al. [26]. Therefore, it becomes imperative to strategically reduce features (bands) efficiently by leveraging feature extraction techniques. As emphasized by Hidalgo et al. [27], this necessity introduces a significant challenge in HSI analysis. The careful balance lies in mitigating computational complexity while acknowledging the potential risk of losing vital information during the feature extraction process, which could significantly impact the ultimate classification results.

Addressing the aforementioned challenges and limitations, this research introduces a novel variant of a feature extraction approach known as spatially coherent local linear embedding (SCLLE). We term this variant supervised spatially spectrally coherent local linear embedding integrated with Wasserstein distance-based graph convolutional network (S^3CLLE-WGCN), which integrates features such as spatial coherence, inspired by Mohan et al. [28], and our proposed supervised method of local linear embedding (LLE) into a GCN framework. This integration results in the S^3CLLE-WGCN methodology. The primary objective of this methodology goes beyond merely enhancing classification accuracy; it also strategically extracts spatial and spectral information to improve classification performance.

Within the framework of the GCN, various forms of Laplacians were implemented utilizing diverse metric functions such as Wasserstein distance (WD), radial-based functions (RBFs), spectral angle mapper (SAM), and spectral information divergence (SID). These metric functions are selected to probe the relationships between pixels and establish a graph-based representation. This approach seeks to discern the most suitable function for graphically representing HSIs, thereby contributing to an informed understanding of its impact on classification performance.

The main contributions are summarized as follows.

1) A supervised spatially spectrally coherent LLE for capturing spatial relationships among neighborhood spectral vectors, simultaneously reducing the dimensionality of the HSI.
2) In order to enhance feature extraction, WD is used to obtain the graph representation of the HSI.

3) A novel adaptation of GCN with WD metric for construction of adjacency matrix for training and classification called Wasserstein graph convolution network (WGCN) is developed.
4) The S^{3}CLLE-WGCN model extracts spatially and spectrally coherent features, simultaneously reduces dimensions, and trains the WGCN for HSI classification.

This article is organized as follows. Section II briefly describes the preliminaries and motivation for this work. Section III explains the proposed S^{3}CLLE-WGCN-GCN methodology with a step-by-step breakdown of the workflow. Section IV presents the experimental setup and a description of the four benchmark datasets. An experimental analysis and discussion of the superior performance of S^{3}CLLE-WGCN on four benchmark datasets for HSI classification is given in Section V. Section VI presents the complexity analysis of the S^{3}CLLE-WGN model, explanations, and illustrations that provide clarity on the contributions of supervised spatially spectrally coherent LLE (S^{3}CLLE) and WGCN. Section VII presents an ablation study of the S^{3}CLLE-WGCN model performance. The conclusions are summarized in Section VIII.

II. Preliminaries

Any deep learning architecture must, directly or indirectly, incorporate modules for dimensionality reduction (DR) or feature extraction to achieve good model performance. Convolutional neural networks inherently perform feature extraction via their convolutional layers, pooling operations, and nonlinear activations. In contrast, GCNs aggregate relational information through the adjacency matrix but lack an intrinsic mechanism to derive node features from raw data. Consequently, for node classification, features must be extracted in a preliminary step. To address this gap, we propose an LLE-based method to learn discriminative node features prior to applying the standard GCN model. A general background on DR and conventional embedding methods is given in this section.

A. Dimensionality Reduction

DR is a technique employed to address the challenge of *curse of dimensionality* in HSIs [29]. The central concept behind DR involves discovering a set of low-dimensional vectors by either linearly or nonlinearly transforming the hyperspectral dataset into a new domain, where informative features are extracted. Given the input $X \in \mathbb{R}^{n\times\mathbb{D}}$ such that $X = [\boldsymbol{x}_1; \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]$, where $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n \in \mathbb{R}^D\}$ feature extraction seeks the low-dimensional embedding space $\widetilde{X} = \{\widetilde{\boldsymbol{x}}_1, \widetilde{\boldsymbol{x}}_2, \ldots, \widetilde{\boldsymbol{x}}_n \in \mathbb{R}^d\}$, where $d \leq D$ (typically $d \ll D$), preserving some information of the classes.

Curse of dimensionality was usually addressed using various DR techniques [30]; principal component analysis (PCA), linear discriminant analysis (LDA), and LLE have been employed to reduce the number of bands in HSIs. While PCA projects data orthogonally, it often struggles with classification tasks when faced with interference sources such as natural background signals and structured, nonrandom noise, including striping [31]. Meanwhile, LDA aims to guide data toward maximum separation between-class means, and LLE is designed to preserve local neighborhood structures. DR methods are also utilized for feature extraction. PCA, LDA, and LLE methods reduce dimensionality as well as extract prominent features that are used for training an ML classifier for HSI classification.

The DR techniques enhance the efficiency of the following HSI processing tasks: mitigating computational complexity on one hand and, on the other hand, enhance the performance of ML models by focusing on significant features, which may yield a higher accuracy level for classification and clustering. In fact, it has been recently used as a preprocessor of a dataset to train a GCN, indicating its relevance in a modern ML pipeline [32].

Feature extraction methods can be divided into two main categories: unsupervised and supervised feature extraction [33], [34], [35]. In this article, we will focus exclusively on the LLE method, both in its unsupervised and supervised forms.

B. Local Linear Embedding

LLE [35], [36] is a nonlinear spectral method for DR utilized for manifold embedding and feature extraction. LLE aims to maintain the local structure of the data when mapping to the embedding space. Data points that are close together in the high-dimensional input space should remain close in the low-dimensional embedding space. By preserving these local relationships, LLE also seeks to ensure that points that are far apart in the input space remain distant from each other in the embedding space. The LLE algorithm can be summarized in three steps, as illustrated in Fig. 1. The steps are as follows.

1) *Compute the k-Nearest Neighbors:* For each data point, identify its k-nearest neighbors using a suitable distance metric. This step establishes the local neighborhood relationships in the dataset.
2) *Compute the Weights That Best Reconstruct Each Data Point From Its Neighbors:* For each data point, calculate the weights that minimize the reconstruction error. This is akin to calculating barycentric coordinates for each point relative to its neighbors. The goal is to preserve the local geometry of the data.
3) *Compute the Low-Dimensional Embedding That Best Preserves the Local Weights:* Using the weights computed in the previous step, find the low-dimensional coordinates of the data points that best preserve the local neighborhood structure. This is typically achieved by solving an eigenvalue problem.

Step 1 is typically done using Euclidean distance to define the neighborhood, although more sophisticated criteria may also be used. For example, Mohan et al. [28] propose a spatially coherent LLE technique that incorporates spatial coherence by comparing pixels based not just on their values but also on their local surrounding structure in the image domain, thereby defining high-dimensional local neighborhoods more accurately. Additionally, various extensions of LLE have been proposed to enhance neighborhood selection, such as using geodesic or weighted distances [37], [38],

adaptive neighborhood sizes [39], and extra processing like neighbor smoothing and short-circuit edge pruning [4]. These methods aim to improve the robustness and accuracy of neighborhood selection, particularly in noisy or sparsely sampled datasets.

In step 2, the weights that best reconstruct each data point from its k neighbors are computed. For each data point, the reconstruction weights are calculated by minimizing the sum of squared reconstruction errors. To determine the reconstruction weight matrix $\mathrm{W} \in \mathbb{R}^{n\times n}$, the task is formulated as an optimization problem aimed at minimizing the sum of squared reconstruction errors

$$\varepsilon(\boldsymbol{W}) = \min_{\mathrm{W}} \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \sum_{j=1}^{k} w_{ij}\boldsymbol{x}_j \right\|^2. \tag{1}$$

This process is akin to calculating barycentric coordinates for each point relative to its neighbors and then minimizing the Euclidean distance between each point and its representation in these barycentric coordinates. The weights are constrained to sum to one, i.e., $\sum_j w_{ij} = 1$ and $\mathrm{W}_{ij} = 0$ if x_i and x_j are not nearest neighbors. These constraints ensure that the reconstruction is invariant to translation, scaling, and rotation, preserving the local geometry of the data.

In step 3, after calculating the reconstruction weights, each sample is mapped from the original high-dimensional feature space to a low-dimensional manifold space. The objective of the embedding is to preserve the local linearity of the feature space. LLE achieves this by selecting the embedding that best reconstructs each sample from its neighbors in the low-dimensional space using the previously computed reconstruction weights. This is formulated as the following minimization problem:

$$\phi(\widetilde{\mathbf{x}}) = \min_{\widetilde{\mathbf{x}}} \sum_{i=1}^{n} \left\| \widetilde{\mathbf{x}}_i - \sum_{j=1}^{k} w_{ij}\widetilde{\mathbf{x}}_j \right\|^2$$
$$\text{subject to } \frac{1}{N}\sum_{i=1}^{n} \widetilde{\mathbf{x}}_i\widetilde{\mathbf{x}}_i^T = I \quad \text{and} \quad \sum_{i=1}^{n} \widetilde{\mathbf{x}}_i = \overrightarrow{0} \tag{2}$$

which represents the low-dimensional coordinates of the data points. Solving this problem typically involves an eigenvalue decomposition, where the desired embedding is obtained from the bottom nonzero eigenvectors of the matrix.

C. Spatial Coherence in Lle

The SCLLE algorithm was studied and presented by Mohan et al. [28]. They extend the classical LLE algorithm by introducing spatial coherence into the computation of high-dimensional neighborhoods. The authors experimentally demonstrated on the Indian Pines dataset that their method effectively reduces the amount of data by more than 75% while improving classification accuracy by approximately 15%.

In their proposed method, pixels are compared based on their local $n_{\text{sur}} \times n_{\text{sur}}$ surrounding patch in the image domain, rather than their individual values. As discussed in the traditional LLE method, the Euclidean distance is typically used to compute the k closest neighbors $\{\boldsymbol{x}_{i,1}, \boldsymbol{x}_{i,2}, \ldots, \boldsymbol{x}_{i,k} \in \mathbb{R}^D\}$ for a point $\vec{x}_i$. Instead, Mohan et al. [28] propose using all the $n_{\text{sur}} \times n_{\text{sur}}$ surrounding pixels and a spatially coherent distance metric to compute the k neighbors in $\mathbb{R}^D$. This approach ensures that the selection of neighbors reflects spatial coherence, while steps 2 and 3 of the LLE algorithm, which involve computing the weights and the low-dimensional embedding, remain unchanged. The new vector $\boldsymbol{x}_i^{\text{new}}$ is constructed by replacing each band of $\boldsymbol{x}_i$ with an n_{sur}^2-dimensional vector formed by its $n_{\text{sur}} \times n_{\text{sur}}$ surrounding pixels, but only for the purpose of identifying the indices of the original vectors $\boldsymbol{x}_i$ that are neighbors. This means that if two pixels $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ are similar, but their respective surrounding patches are dissimilar, the distance between the new pixel vectors $\boldsymbol{x}_i^{\text{new}}$ and $\boldsymbol{x}_j^{\text{new}}$ will be large. Consequently, these original pixels will not be considered neighbors and will not be used together to estimate the LLE. The new distance metric, which incorporates the spatial coherence between pixel vectors, ensures that the LLE algorithm accounts for the local spatial context of each pixel. This approach improves the robustness and accuracy of the DR, leading to better classification and segmentation results in HSIs.

D. Supervised Lle

De Ridder et al. [34] and Zhang [40] proposed supervised versions of LLE. Supervised LLE (SLLE) addresses the issue of disjoint manifolds in multiclass datasets by redefining the distance metric for neighborhood selection to account for class information, ensuring larger distances between points from different classes despite their close Euclidean proximity.

1) De Ridder et al.'s Method for Classification: De Ridder et al. [34] introduced an SLLE method aimed at addressing the issue of disjoint manifolds in multiclass datasets. They proposed modifying the distance matrix to artificially increase the interclass distances while keeping the intraclass distances unchanged. The modified distance matrix is defined as

$$\Delta' = \Delta + \alpha \max(\Delta)\Lambda, \quad \alpha \in [0, 1] \tag{3}$$

where $\Lambda_{ij} = 1$ if x_i and x_j belong to the same class and 0 otherwise. When $\alpha = 0$, the method reduces to the original unsupervised LLE. When $\alpha = 1$, the method fully separates the classes, ensuring that neighbors are selected from the same class only. This modification enhances the embedding by separating classes in the low-dimensional space [34].

2) Zhangs Method for Classification: Zhang [40] proposed an enhanced version of SLLE (ESLLE) that not only increases the interclass variances but also reduces the intraclass variances. The distance modification in ESLLE is given by

$$D'_{ij} = \begin{cases} \sqrt{1 - e^{-D_{ij}^2/\beta}}, & \text{if } c_i = c_j \\ \sqrt{e^{D_{ij}^2/\beta} - \alpha}, & \text{otherwise} \end{cases} \tag{4}$$

where β is a parameter representing the average distance between all pairs of data points. This approach ensures that points from different classes are separated more distinctly, and points from the same class are clustered more tightly in the embedding space.

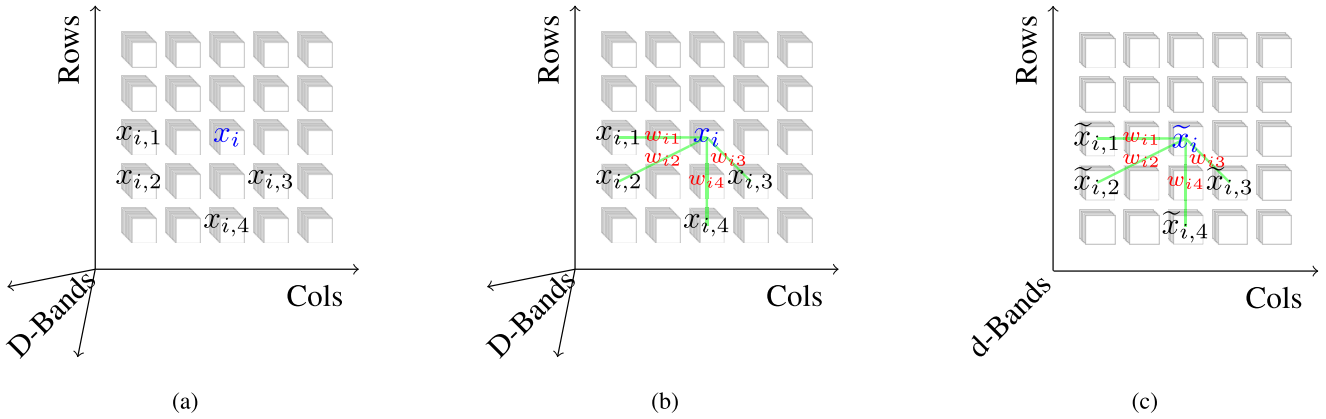


Fig. 1. Steps in the LLE method for mapping high-dimensional data to a low-dimensional space. (a) Identifying the k-nearest neighbors, (b) performing linear reconstruction using these neighbors, and (c) achieving linear embedding with the computed weights.

3) Boucher et Al.'s Method for Regression: Boucher et al. [41] introduced a method for SLLE that is specifically designed for regression problems. Their approach, called manifold learning for regression (MLR), modifies the objective function in the second step of LLE to account for the response variables. The modified objective function is

$$\varepsilon(\boldsymbol{W}) = \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \sum_{j=1}^{K} w_{ij} \boldsymbol{x}_j \right\|^2 + \frac{\mu}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} k_{ij} \left\| \boldsymbol{x}_i - \boldsymbol{x}_j \right\|^2 \quad (5)$$

where μ is a regularization parameter and k_{ij} is the similarity score between the response variables. This approach ensures that samples with similar responses are embedded closer together, improving the regression performance. However, this method is tailored for regression tasks rather than classification.

E. GCN for HSI Classification

Recently, there has been a growing interest in the utilization of GCNs for diverse applications. Notably, GCN has demonstrated efficacy in image processing, particularly in scenarios involving nonregular regions. Its capacity to encapsulate a broader spatial and spectral relationship between pixels, surpassing the capabilities of CNN, has been a driving factor in its exploration [24].

The application of GCN in HSI processing and classification has gained prominence since 2018. Farideh et al. [42] were among the early adopters, introducing a method that effectively represents data residing on smooth manifolds through the creation of a semi-supervised adjacency matrix. This matrix is constructed using a limited number of labeled pixels.

Qin et al. [43] introduced the S^2GCNs method, incorporating not only the spectral aspect but also leveraging the spatial information of each pixel during the graph convolution process. A notable challenge associated with GCN lies in the computational cost required for adjacency matrix construction. In response to this, Hong et al. [24] proposed a batch-oriented approach for constructing the adjacency matrix during model training. They utilized the k-nearest neighbors of each pixel for this purpose and explored hybrid methods involving the fusion of GCN with CNN.

The exploration of superpixels in the context of GCN gained traction between 2021 and 2022. Jia et al. [25] proposed the graph-in-graph (GiG) model, which leverages superpixels as internal and external graphs based on spatial adjacent relationships. Concurrently, Gong et al. [44] also reported similar work, constructing the adjacency matrix using superpixels. The construction of the adjacency matrix is a crucial aspect of GCN, serving as the means to appropriately represent image data for the GCN model.

Orozco et al. [15] proposed a dynamic method for constructing the adjacency matrix, introducing variability in the neighborhood associated with each pixel based on its class affiliation. This dynamic approach positively impacted classification accuracy, indicating the potential for adaptive strategies in adjacency matrix construction within the GCN framework.

To comprehend the fundamental principles of GCNs, exploring the mathematical foundations that enable their use in semi-supervised learning on graph-structured data is essential. GCNs expand the idea of convolutions to graph data by utilizing spectral graph theory. This involves defining the graph Laplacian, a key operator derived from the adjacency matrix that represents the graph's structure. The normalized graph Laplacian is calculated as follows:

$$L = I_n - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \quad (6)$$

where D is a diagonal matrix representing node degrees, D_{ii} represents the number of edges connected to node i, and A is the adjacency matrix. The normalized Laplacian matrix is positive semi-definite, and with this property, (6) can be written as $U\Lambda U^T$, where U is a matrix of eigenvectors and Λ is a diagonal matrix of eigenvalues λ. The eigenvectors conform to the orthonormality condition $UU^T = I$. The eigenvectors and eigenvalues of this Laplacian form the graph Fourier basis, facilitating the transformation of graph signals into the frequency domain. The graph Fourier transform of a signal X is defined as $\mathcal{F}(X) = U^T X$, and its inverse is $\mathcal{F}^{-1}(X) = UX$, where U is the matrix of eigenvectors. The graph convolution with a filter g is defined as

$$X * g = \mathcal{F}^{-1}\left(\mathcal{F}(X) \odot \mathcal{F}(g)\right) \quad (7)$$

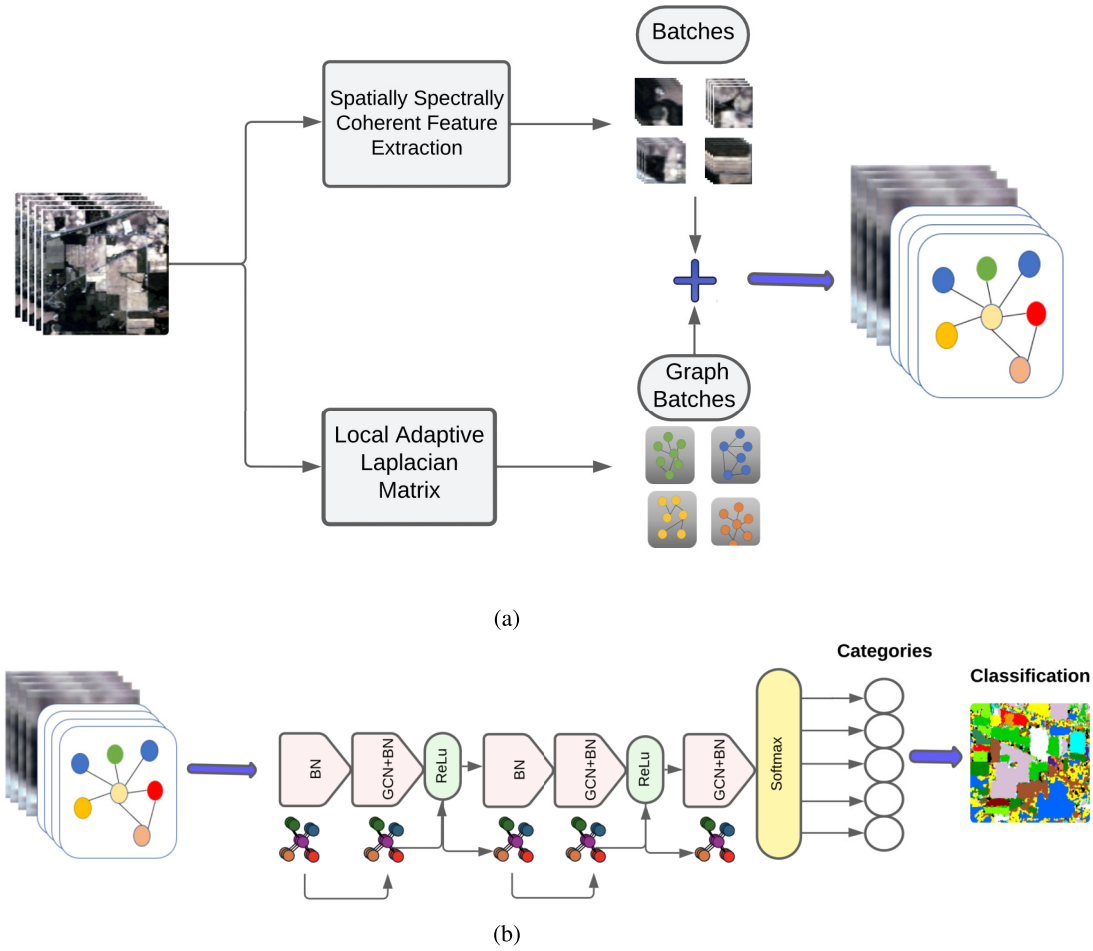


Fig. 2. Overview of S^{3}CLLE-WGCN architecture for HSI classification. (a) S^{3}CLLE feature extraction and graph creation. (b) WGCN workflow.

where $\odot$ is the element-wise product and X is a feature vector of all nodes of a graph. Reducing this equation to

$$X * g = UgU^T X \tag{8}$$

where g is typically parameterized as a function of the Laplacian eigenvalues and is usually defined as $g = \text{diag}(U^T g)$. To address the computational complexity of eigenvector decomposition in (6) Hammond et al. [45] propose approximating g using a truncated expansion of Chebyshev polynomials up to the Kth order

$$g = \sum_{i=0}^{k} \theta_i T_i\left(\hat{\Lambda}\right) \tag{9}$$

where $\hat{\Lambda} = \frac{2}{\lambda_{\max}}\Lambda - I_n$. By taking $k = 1$ and the largest eigenvalue $\lambda_{\max}$ to 2, (3) can be written as [46]

$$X * g = \theta\left(I_n + D^{-\frac{1}{2}} A D^{-\frac{1}{2}}\right) X. \tag{10}$$

Given that $I_n + D^{-(1/2)} A D^{-(1/2)}$ has eigenvalues within the range $[0, 2]$, repeatedly applying this operator can result in numerical instabilities. This can cause the explode/vanish gradients when used in a deep neural network model. To mitigate this issue, the *renormalization trick* proposed in [46] is typically used, where $I + D^{-(1/2)} A D^{-(1/2)} \rightarrow \hat{D}^{-(1/2)} \hat{A} \hat{D}^{-(1/2)}$, with $\hat{A} = A + I$ and $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. The propagation rule for GCNs then becomes

$$H_{l+1} = \sigma\left(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} + b^{(l)}\right) \tag{11}$$

where matrix $H^{(l)}$ denotes the features output in the lth layer, σ is the activation function, and $W^{(l)}$ and $b^{(l)}$ are the learned weights and biases.

III. Methodology

This section presents a detailed description of the novel S^{3}CLLE-WGCN model. The first section focuses on feature extraction using the supervised spatially spectrally coherent LLE feature extraction. This is followed by a description of the WGCN that includes the construction of the adjacency matrix using WD, as depicted in Fig. 2(a). The second section, illustrated in Fig. 2(b), outlines the workflow of the graph convolutional model employed for the classification of HSIs.

A. Supervised Spatially Spectrally Coherent Lle (S^{3}CLLE)

In this work, we adapt the concept from Boucher et al.'s MLR method [41] to address classification problems. Specifically, we introduce a penalty term to the loss function in the third term of LLE to ensure that points belonging to the same

Algorithm 1 S^{3}CLLE Algorithm

Input: $(X, Y) = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_n, y_n) \in \mathbb{R}^D \times \text{Labels}\}$; n_{sur}

Output: $\widetilde{X} = \{\widetilde{\mathbf{x}}_1, \widetilde{\mathbf{x}}_2, \ldots, \widetilde{\mathbf{x}}_n \in \mathbb{R}^d\}$, where $d \ll$ D

1 Finding K neighbors for every pixel $\boldsymbol{x}_i$ *using spatial coherence surrounding*, $\{\boldsymbol{x}_{i,1}, \boldsymbol{x}_{i,2}, \ldots, \boldsymbol{x}_{i,K} \in \mathbb{R}^D\}$;

2 Computing local reconstruction weighted matrix $\boldsymbol{W}$ by minimizing cost function:

$$\varepsilon(\boldsymbol{W}) = \sum_{i=1}^{n} \left\| \boldsymbol{x}_i - \sum_{j=1}^{K} w_{ij} \boldsymbol{x}_{i,j} \right\|^2;$$

3 Mapping the original dataset to d-dimensional manifold coordinates. Finding the vectors $\widetilde{\mathbf{x}}_i$ to minimize the cost function:

4 $$\phi(\widetilde{X}) = \frac{1}{2} \sum_{i=1}^{n} \left\| \widetilde{\mathbf{x}}_i - \sum_{j=1}^{K} w_{ij} \widetilde{\mathbf{x}}_j \right\|^2 + \frac{\mu}{2} \sum_{i=1}^{n} \sum_{j=1}^{K} \left\| \widetilde{x}_i - \widetilde{x}_j \right\|^2 k_{ij}$$

5 subject to

$$\frac{1}{N} \sum_{i=1}^{n} \widetilde{\mathbf{x}}_i \widetilde{\mathbf{x}}_i^T = I \quad \text{and} \quad \sum_{i=1}^{n} \widetilde{\mathbf{x}}_i = \vec{0}$$

class are mapped closer together, as shown in Algorithm 1. The modified loss function is

$$\phi(\widetilde{X}) = \sum_{i=1}^{n} \left\| \widetilde{\mathbf{x}}_i - \sum_{j=1}^{K} w_{ij} \widetilde{\mathbf{x}}_j \right\|^2 + \frac{\mu}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} k_{ij} \left\| \widetilde{\mathbf{x}}_i - \widetilde{\mathbf{x}}_j \right\|^2 \tag{12}$$

where

$$k_{i,j} = \begin{cases} 1 & , \text{if} \quad y_i = y_j \\ \gamma \ll 1, & \text{if} \quad y_i \neq y_j \end{cases}$$

$$\text{subject to } \frac{1}{N} \sum_{i=1}^{n} \widetilde{\mathbf{x}}_i \widetilde{\mathbf{x}}_i^T = I \quad \text{and} \quad \sum_{i=1}^{n} \widetilde{\mathbf{x}}_i = \vec{0} \tag{13}$$

where μ is a regularization parameter. The constraint $\sum_i \widetilde{x}_i = \mathbf{0}$ ensures that the embedding is invariant to translation, and the constraint $\widetilde{X}^T \widetilde{X} = I$ ensures that the problem is well-posed. Through this modification, points of the same class are embedded closer together, thereby improving the classification accuracy. If we consider $\gamma = 0$, the regularization term becomes

$$\phi(\widetilde{X}) = \sum_{i=1}^{N} \left\| \widetilde{\mathbf{x}}_i - \sum_{j=1}^{K} w_{ij} \widetilde{\mathbf{x}}_j \right\|^2 + \mu \sum_{(i,j) \in \mathcal{C}} \left\| \widetilde{\mathbf{x}}_i - \widetilde{\mathbf{x}}_j \right\|^2 \tag{14}$$

where μ is a regularization parameter and $\mathcal{C}$ is the set of pairs of points belonging to the same class.

B. Wasserstein GCN

1) Design of the Adjacency Matrix: In graph convolutional methods, a central focus lies in the graph-based representation of HSIs using their adjacency matrix. This representation facilitates the propagation and aggregation of features across graph connections, thereby enabling the capture of nonlocal dependencies and intricate structures [46]. The adjacency matrix plays a crucial role in modeling diverse pixel relationships, including spatial proximity, intensity similarity, and spectral characteristics in hyperspectral imagery [14], [47].

For the construction of the adjacency matrix A, the adaptive method proposed in [15] was implemented for selecting neighbors for each pixel. To construct the adjacency matrix, each pixel is iterated over with a selected neighborhood radius. With the neighborhood radius defined, the dispersion among the pixels enclosed within this radius R is calculated using the following equation:

$$S^2 = \frac{\sum_j (\boldsymbol{x}_i - \bar{\boldsymbol{x}})}{n - 1} \tag{15}$$

where $\boldsymbol{x}_i$ is the pixel intensity value per band, $\bar{\boldsymbol{x}}$ is the average pixel intensity inside R, and n is the number of pixels.

Following the dispersion calculation, the decision to adjust the radius is based on a threshold value derived from the average of a set of pixels belonging to the same class. Once the threshold condition is satisfied, the elements of A are computed. The WGCN makes use of the WD to compute each element of the adjacency matrix A. The WD is also known as the optimal transport metric. It is a measure that quantifies the dissimilarity between two probability distributions [48]. This metric is particularly useful in the analysis of complex data, such as HSIs, due to its ability to consider the geometric structure of distributions in space. The main advantage of using the WD compared to other metrics is its robustness to small variations and noise, enabling better differentiation between similar pixels. Mathematically, the first-order WD between two probability distributions P and Q is defined as

$$d_W(P, Q) = \inf_{\gamma \in \Pi(P,Q)} \mathbb{E}_{(x_i, x_j))} \left[d(x_i, x_j) \right] \tag{16}$$

where $\Pi(P, Q)$ is the set of all joint distributions $\gamma(x_i, x_j)$ whose marginals are P and Q, and $d((x_i, x_j)$ is the distance between points x and y in the distribution space.

C. Adjacency Matrix Design for GCN

The design of the adjacency matrix is a crucial element in implementing GCNs for HSI classification. This matrix encodes the relationships between pixels, facilitating the propagation and aggregation of features across the graph. Here, we explore various methods for constructing the adjacency matrix, each utilizing different metrics to capture pixel similarities. We discuss the radial basis function (RBF), spectral angle mapping (SAM), SID, and our proposed WD method. These approaches aim to enhance the representation of HSIs within the GCN framework, ultimately improving classification accuracy and robustness.

1) RBF Adjacency Matrix: The RBF is employed to create the adjacency matrix by leveraging the Euclidean distance between pixels. The RBF adjacency matrix, A_{RBF}, is defined as

$$A_{\text{RBF}}(i, j) = \exp\left(\frac{-\left\| \mathbf{x}_i - \mathbf{x}_j \right\|^2}{\sigma^2} \right) \tag{17}$$

Algorithm 2 Adjacency Matrix for S^{3}CLLE-WGCN

Input: Manifold S^{3}CLLE HSI, i.e $\widetilde{X} = \{\widetilde{x}_1, \widetilde{x}_2, \cdots, \widetilde{x}_n \in \mathbb{R}^d\}$;
for $\widetilde{x} \in \widetilde{X}$ **do**
  Set a radius **R** for pixel $\widetilde{x}$ in space image domain;
  Find Homogeneity of embedding pixels inside **R** using dispersion;
  while *dispersion* > *threshold value* **do**
    Decrease R until meet the desired threshold;
    if $R < 1$ **then**
      Set minimum possible radius to 1;
      stop while;
    end
  end
  Define Neighbor of $\widetilde{x}$ is embedding pixel withing the radio < **R**;
end
Calculate the Wasserstein similarity matrix using Equation (23);
Calculate the Wasserstein Adjacency matrix A_{WD} using Equation (26);
Output: Wasserstein Adjacency Matrix, i.e, A_{WD}

where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the vectors of spectral signatures associated with pixels i and j, respectively. The parameter σ^2 controls the "width" or "scale" of the RBF, determining the influence range of each pixel. This matrix effectively captures the similarity between pixels, with higher values indicating greater similarity, and is usually utilized to form the adjacency matrix in the GCN.

2) SAM Adjacency Matrix: The SAM method calculates the spectral angle between pixels using their spectral vectors. This algorithm compares the angle between each pixel vector in the d-dimensional space and the endmember spectrum, with smaller angles indicating a closer match to the reference spectrum. The elements of the SAM adjacency matrix, A_{SAM}, are defined by the spectral angle metric [49] as

$$A_{\mathrm{SAM}}(i,j) = \cos^{-1}\left(\frac{\sum_{k=1}^{d} x_k y_k}{\sqrt{\sum_{k=1}^{d} x_k^2}\sqrt{\sum_{k=1}^{d} y_k^2}}\right) \tag{18}$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ and $\mathbf{y} = (y_1, y_2, \ldots, y_d)$ are the spectral signature vectors of pixels i and j, respectively, and d is the number of spectral bands.

Although SAM values do not range between 0 and 1, this is not a limitation because the angle is proportional to the similarity between vectors. The SAM value tends to zero for similar vectors and approaches $\pi/2$ for dissimilar vectors. This essentially simulates an unnormalized adjacency matrix, but the issue can be resolved by normalizing the matrix.

3) SID Matrix: The SID measure uses the distance between two-pixel vectors x_i and x_j in terms of their respective probability mass functions to determine the degree of spectral similarity. According to [50], the elements of the SID adjacency matrix, A_{SID}, are defined as

$$A_{\mathrm{SID}}(i,j) = D\left(x_i \parallel x_j\right) + D\left(x_j \parallel x_i\right) \tag{19}$$

Algorithm 3 Pseudocode of S^{3}CLLE-WGCN for HSI Classification

Input: Original HSI i.e $X \in \mathbb{R}^{n\times D}$, Ground Truth i.e $Y \in \mathbb{R}^n$, $n_{\mathrm{sur}} = 3$, d, $k = 8$, epochs $= 200$, batch size $= 100$
Split the HSI data into labeled and unlabeled data;
$X_{\mathrm{Label}} = \{x_{l1}, x_{l2}, \cdots, x_{ls}\}$;
$X_{\mathrm{No\ Label}} = \{x_{nl1}, x_{nl2}, \cdots, x_{nlm}\}$;
Define a subset of the labeled data with their respective labels to fit the embedding;
$X_{\mathrm{embedd}} \subset X_{\mathrm{Label}}$;
Fit the S^{3}CLLE model to the data $(X_{\mathrm{embedd}}, Y_{\mathrm{embedd}})$ with parameters $n_{\mathrm{sur}} = 3, k = 8$;
Transform the data $X \in \mathbb{R}^{n\times D}$ with the fit S^{3}CLLE model;
$\widetilde{X} = \mathrm{S}^3\mathrm{CLLE}(X) \in \mathbb{R}^{n\times d}$;
Compute $A_{\mathrm{WD}} \in \mathbb{R}^{n\times n}$;
Divide $\widetilde{X}_{\mathrm{Label}}$ into Training and Test sets;
$\widetilde{X}_{\mathrm{Train}}, \widetilde{X}_{\mathrm{Test}} \leftarrow \mathrm{split}(\widetilde{X}_{\mathrm{Label}})$;
Process the training data with the GCN method in batches;
Initialize **W** and **b** parameters;
for *each batch of pixels* **do**
  $\hat{H}^{l+1} = \sigma(\hat{D}^{-\frac{1}{2}}\hat{A}_{\mathrm{WD}}\hat{D}^{-\frac{1}{2}}H^{(l)}W^{(l)} + b^{(l)})$;
  Return $\hat{H}_b^{l+1}$, Loss;
  Softmax;
  Optimize loss function using Adam;
  Update parameters;
Output: Predicted label for each pixel using argmax;

where

$$D\left(x_j \parallel x_i\right) = \sum_{k=1}^{d} p_k\left(I_k\left(x_j\right) - I_k\left(x_i\right)\right) \tag{20}$$

is called the relative entropy and measures the discrepancy in the self-information of the band k in x_i relative to the self-information of band k in x_j and

$$D\left(x_i \parallel x_j\right) = \sum_{k=1}^{d} q_k\left(I_k\left(x_i\right) - I_k\left(x_j\right)\right) \tag{21}$$

is the discrepancy measure of x_i relative to x_j. Here, p_k and q_k are the probability mass functions of the spectral bands, and $I_k(\cdot)$ denotes the self-information function of the band k, defined as

$$I_k(x) = -\log\left(p_k(x)\right) \tag{22}$$

where $p_k(x)$ is the probability of observing the value x in the band k.

4) WD Adjacency Matrix: We start by defining the Wasserstein similarity matrix as

$$W(i,j) = \begin{cases} 0, & \text{if } i = j \\ S_{ii} - d_W\left(x_i, x_j\right), & \text{if } j\text{is neighbor of } i \\ 0, & \text{if if } j\text{is not neighbor of } i \end{cases} \tag{23}$$

TABLE I
CLASSIFICATION PERFORMANCE OF VARIOUS GCN METHODS FOR THE INDIAN PINES DATASET

Class No.	$F^2HNNssc$ [23]	SSOGCN[51]	GCN[23]	Non-local GCN[52]	MiniGCN[23]	AN-GCN[15]	**S^{3}CLLE-WGCN**
1	95.59	20.95	53.54	89.03	72.54	85.04	**98.77**
2	98.92	87.52	53.01	**100.00**	55.99	81.76	99.74
3	**100.00**	93.46	87.77	93.51	92.93	95.65	98.91
4	**100.00**	**100.00**	90.89	94.12	92.62	88.59	96.87
5	76.44	98.38	87.95	98.18	94.98	96.84	**100.00**
6	**100.00**	98.68	97.97	78.78	98.63	99.54	99.09
7	96.50	**100.00**	53.81	99.38	64.71	91.94	95.53
8	98.33	**100.00**	54.99	**94.94**	68.78	81.39	**99.63**
9	95.79	**100.00**	38.28	**97.27**	69.33	90.07	94.50
10	**100.00**	92.41	98.05	**100.00**	98.77	**100.00**	**100.00**
11	**99.92**	90.19	84.58	97.44	87.78	94.61	97.67
12	**100.00**	95.76	65.80	**100.00**	50.00	90.61	99.39
13	**100.00**	99.35	97.85	**100.00**	**100.00**	**100.00**	**100.00**
14	97.83	97.12	91.30	83.09	48.72	**100.00**	66.67
15	81.70	99.11	85.71	88.24	72.73	**100.00**	**100.00**
16	50.00	**100.00**	**100.00**	86.70	80.00	**100.00**	80.00
OA(%)	96.25	92.51	65.97	87.92	75.11	88.51	**95.49**
AA (%)	93.19	92.06	77.54	**93.79**	78.03	93.50	**98.44**
k	0.9514	91.45	0.6184	0.8625	0.7164	0.8692	**0.982**

TABLE II
CLASSIFICATION PERFORMANCE OF VARIOUS GCN METHODS FOR THE HOUSTON UNIVERSITY DATASET

Class No.	MiniGCN[53]	DIGCN[53]	DRGCN[54]	GCN[46]	CAD-GCN [55]	MiniGCN[24]	AN-GCN[15]	**S^{3}CLLE-WGCN**
1	94.85±3.58	93.07±2.73	82.8	88.16±1.90	94.45±3.49	98.39	**100.00**	**100.00**
2	98.35±1.71	94.17±2.93	93.38	97.20±0.48	96.43±2.83	92.11	98.34	**100.00**
3	98.09±1.74	95.00±1.68	98.95	97.91±0.13	95.17±4.11	99.60	**100.00**	98.02
4	95.60±2.13	90.47±4.09	90.03	96.55±0.41	94.82±2.38	96.78	99.62	**99.91**
5	98.64±0.72	100.00±0.00	97.02	89.79±0.71	98.91±1.51	97.73	**100.00**	**100.00**
6	96.58±1.80	94.10±3.86	98.30	98.21±1.15	97.48±3.48	95.10	**100.00**	**100.00**
7	76.05±1.53	96.06±2.80	88.77	73.67±1.94	91.58±3.16	57.28	95.94	**97.26**
8	77.28±3.75	73.36±5.63	80.06	65.71±4.64	74.63±4.82	68.09	97.06	**99.53**
9	78.98±2.24	94.33±3.33	94.18	70.27±3.03	86.75±3.58	53.92	91.76	**100.00**
10	82.92±3.80	88.76±7.63	**99.66**	74.71±2.32	94.24±3.34	77.41	99.23	99.42
11	70.07±3.69	90.68±4.32	97.42	75.36±2.37	94.65±2.73	84.91	97.97	**99.52**
12	85.87±3.99	87.08±4.25	91.93	79.29±4.80	89.55±1.93	77.23	**97.98**	97.31
13	80.93±2.57	92.79±4.34	84.51	12.09±2.68	96.80±3.68	50.88	90.88	**99.65**
14	97.73±1.87	**100.00±0.00**	**100.00**	86.03±3.31	**100±0.00**	98.38	**100.00**	**100.00**
15	99.04±0.76	97.90±1.62	95.07	95.29±1.67	98.02±1.42	98.52	**100.00**	99.58
OA(%)	87.00±0.71	91.72±0.64	92.15	80.35±0.61	92.51±0.73	81.71	97.88	**99.28**
AA (%)	88.73±0.58	92.52±0.16	92.80	80.02±0.46	93.57±0.60	83.09	97.92	**99.35**
k	0.8594±0.077	0.9103±0.069	0.9151	0.7872±0.066	0.9189±0.078	0.8018	0.9770	**0.992**

where S_{ii} is the sum of the WDs from pixel i to its nearest neighbors, specifically

$$S_{ii} = \sum_{j \in \text{neighbors of } i} d_W\left(x_i, x_j\right). \tag{24}$$

This implies that S_{ii} represents the sum of the distances from pixel i to its adaptive nearest neighbors based on the WD. Notably, this matrix does not account for the self-similarity of pixel i but ensures that if two vectors i and j have a smaller WD compared to pixels i and k, then their similarity measure $S_{ii} - d_W(i, j)$ is larger. Although this similarity is not normalized, it does not pose a problem as we can normalize it subsequently. We then define the normalized similarity matrix as

$$\hat{W}(i, j) = S^{-1/2} W S^{-1/2} \tag{25}$$

where S is a diagonal matrix such that $S(i, i) = S_{ii}$.

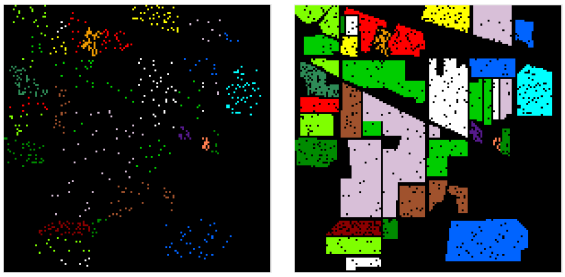
Finally, we define the adjacency matrix as

$$A_{\text{WD}} = I + \hat{W} = I + S^{-1/2} W S^{-1/2}. \tag{26}$$

D. GCN Architecture

The proposed S^{3}CLLE-WGCN method is inspired by the MiniGCN approach [24]. In this method, batches of pixels are extracted similar to classical deep learning models like CNNs, and subgraphs are constructed for each batch. The propagation rule for these subgraphs follows the general rule applied to each batch:

$$\hat{H}_{l+1}^{b_i} = \sigma\left(\hat{D}^{b_i}{}^{-\frac{1}{2}} \hat{A}_{\text{WD}}^{b_i} \hat{D}^{b_i}{}^{-\frac{1}{2}} H^{b_i(l)} W^{b_i(l)} + b^{b_i(l)}\right) \tag{27}$$



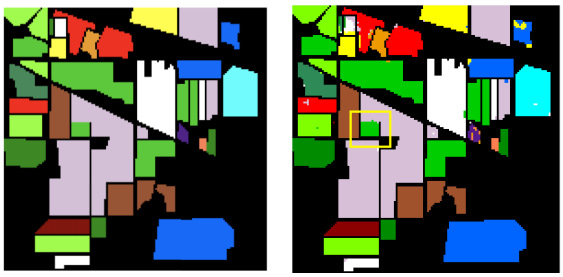
(a) (b)

Fig. 3. (a) Training and (b) testing data for the Indian Pines dataset.

TABLE III

NUMBER OF TRAINING AND TESTING SAMPLES FOR THE DIFFERENT CLASSES IN THE INDIAN PINES DATASET

Class No.	Class Name	Training	Testing
1	Corn Notill	50	1384
2	Corn Mintill	50	784
3	Corn	50	184
4	Grass pasture	50	447
5	Grass trees	50	697
6	Hay Windrowed	50	439
7	Soybean Notill	50	918
8	Soybean Mintill	50	2418
9	Soybean clean	50	564
10	Wheat	50	162
11	Woods	50	1244
12	Buildings Grass Trees Drives	50	330
13	Stone Steel Towers	50	45
14	Alfalfa	15	39
15	Grass Pasture Mowed	15	11
16	Oats	15	5
	total	695	9671



(a) (b)

Fig. 4. Classification map for Indian Pines dataset. (a) Ground-truth map. (b) S^{3}CLLE-WGCN.

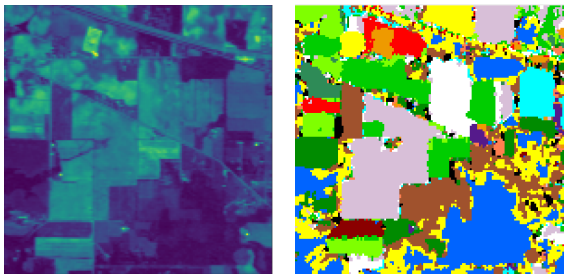
where b_i represents the batches of pixels or subgraphs used for network training. The final output of the propagation rule is a concatenation of all the subgraph outputs

$$\hat{H}_{l+1} = \left[H^{b_1}_{(l+1)}, H^{b_2}_{(l+1)}, H^{b_3}_{(l+1)}, \ldots, H^{b_N}_{(l+1)}\right]. \quad (28)$$

The pseudocode for creating the adjacency matrix for this method is described in Algorithm 2, and the pseudocode for the overall method is provided in Algorithm 3. Fig. 2 illustrates the architecture of the proposed approach.

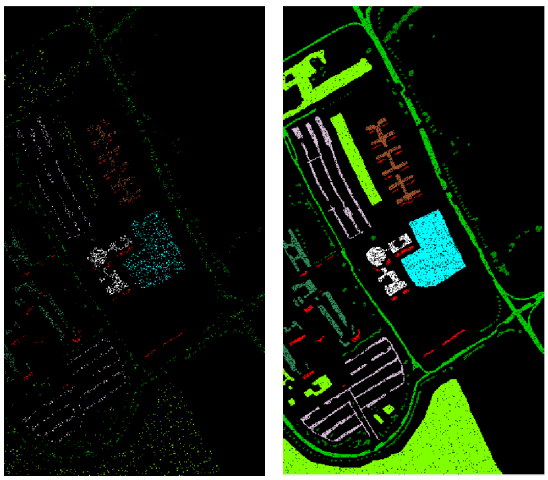
IV. EXPERIMENTAL SETUP AND BENCHMARK DATASETS

In this section, we detail the experimental setup for the S^{3}CLLE-WGCN architecture, which has been implemented using Python. We also provide a comprehensive description of the four benchmark HSI datasets: Indian Pines, Pavia, Houston, and Salinas, which are utilized in our experiments.



(a) (b)

Fig. 5. Classification map for the whole Indian Pines dataset. (a) Indian Pines HSI. (b) S^{3}CLLE-WGCN.



(a) (b)

Fig. 6. (a) Training and (b) testing data in the Pavia dataset.

TABLE IV

NUMBER OF TRAINING AND TESTING SAMPLES FOR THE DIFFERENT CLASSES IN THE PAVIA DATASET

Class No.	Class Name	Training	Testing
1	Asphalt	548	6083
2	Meadows	540	18109
3	Gravel	392	1707
4	Trees	524	2540
5	Painted metal sheets	265	1080
6	Bare Soil	532	4497
7	Bitumen	375	955
8	Self-Blocking Bricks	514	3168
9	Shadows	231	716
	Total	3921	38855

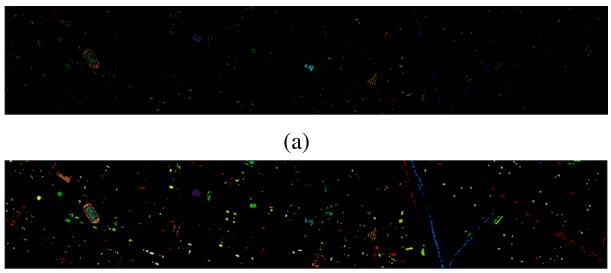
A. S^{3}CLLE-WGCN Method Configuration

The S^{3}CLLE-WGCN method's embedding part employs specific settings to ensure optimal performance. The neighborhood size is fixed at $n_{\text{sur}} = 3$, and 80% of the available training data are allocated for embedding purposes. We consider $k = 8$ nearest neighbors for embedding, aiming to reduce the dimensionality of all HSIs to a target dimension $d = 100$.



(a) (b)

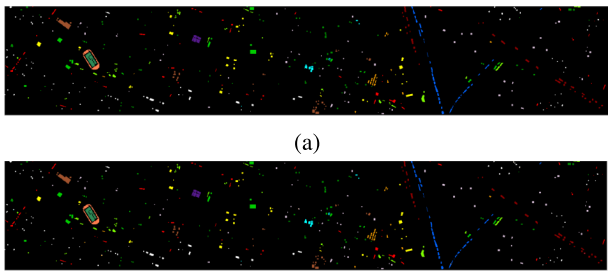
Fig. 7. Classification map in the Pavia dataset. (a) Ground-truth map. (b) S^{3}CLLE-WGCN.



(a)

(b)

Fig. 8. (a) Training and (b) testing data for Houston University dataset.



(a)

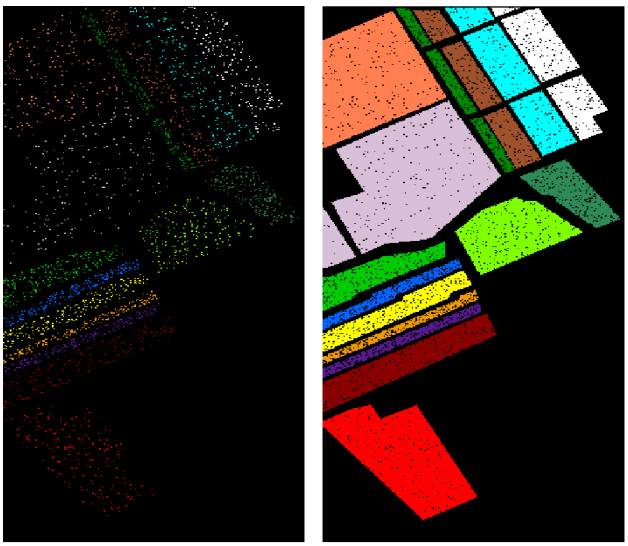
(b)

Fig. 9. Classification map for Houston University dataset. (a) Ground-truth map. (b) S^{3}CLLE-WGCN.

For constructing the adaptive adjacency matrix, following the methodology described in [15], we select a neighborhood radius $R = 4$ for each neighbor. Initially, approximately 44 nearest neighbors are chosen. To ensure the quality and relevance of the neighbors, we apply dispersion thresholds specific to each dataset: 0.16 for Indian Pines, 0.03 for Pavia, 0.24 for Houston, and 0.06 for Salinas. These thresholds are derived from the average variance observed in pure classes. Additionally, the minimum number of selected neighbors is set to four closest neighbors.

B. GCN Model Hyperparameter Settings

The GCN model used in this study is configured with the following hyperparameters: a learning rate of 0.001, a batch size of 200, and 400 epochs for training. The workflow of the GCN model is illustrated in Fig. 2. The architecture includes two layers of batch normalization (BN), which are crucial for stabilizing the training process. These BN layers are supplemented with three additional layers of GCN and BN. The model is optimized using the Adam optimizer, and the loss function employed is the softmax cross entropy with logits.



(a) (b)

Fig. 10. (a) Training and (b) testing data for the Salinas Scene dataset.

C. Benchmark Datasets

We utilize four benchmark datasets for the classification experiments: Indian Pines, Pavia, Houston, and Salinas. These datasets are well-known in the hyperspectral imaging community and provide a robust basis for evaluating the performance of the proposed S^{3}CLLE-WGCN architecture. The detailed descriptions of the training and testing samples for each dataset are provided in Tables III–VI.

This section presents the experimental setting of the S^{3}CLLE-WGCN architecture, which is implemented in Python. The four benchmark datasets, Indian Pines, Pavia, Houston, and Salinas HSIs, are also described here. For the S^{3}CLLE-WGCN method, the following settings are used for the embedding part: a neighborhood size of $n_{\text{sur}} = 3$, 80% of the training data are used for embedding, $k = 8$ nearest neighbors are considered, and the target dimensionality is $d = 50$ for all HSIs. For constructing the adaptive adjacency matrix based on [15], a neighborhood radius of $R = 4$ is chosen for each neighbor. The initial number of nearest neighbors selected is approximately 44. Dispersion thresholds of 0.16 for Indian Pines, 0.03 for Pavia, 0.24 for Houston, and 0.06 for Salinas are applied. These thresholds are determined based on the average variance of pure classes. The minimum number of selected neighbors is set to four closest neighbors. The hyperparameter settings for the GCN model are: a learning rate of 0.001, with a batch size of 200, and 400 epochs for training. The GCN workflow is illustrated in Fig. 2. Two layers

TABLE V
NUMBER OF TRAINING AND TESTING SAMPLES FOR THE DIFFERENT CLASSES IN THE HOUSTON UNIVERSITY DATASET

Class No.	Class Name	Training	Testing
1	Healthy grass	198	1053
2	Stressed grass	190	1064
3	Synthetic grass	192	505
4	Tree	188	1056
5	Soil	186	1056
6	Water	182	143
7	Residential	196	1072
8	Commercial	191	1053
9	Road	193	1059
10	Highway	191	1036
11	Railway	181	1054
12	Parking lot 1	192	1041
13	Parking lot 2	184	285
14	tennis court	181	247
15	Running track	187	473
	Total	2832	12197

TABLE VI
NUMBER OF TRAINING AND TESTING SAMPLES FOR THE DIFFERENT CLASSES IN THE SALINAS SCENE DATASET

Class No.	Class Name	Training	Testing
1	Brocoli green weeds 1	200	1809
2	Brocoli green weeds 2	200	3526
3	Fallow	200	1776
4	Fallow Rough Plow pasture	200	1194
5	Falow Smooth trees	200	2478
6	Stubble	200	3759
7	Celery	200	3379
8	Grapes Untrained	200	11071
9	soil Vinyard Develop	200	6003
10	Corn Senesced Green Weeds	200	3078
11	Lettuce Romaine 4wk	200	868
12	Lettuce Romaine 5wk	200	1727
13	Lettuce Romaine 6wk	200	716
14	Lettuce Romaine 7wk	200	870
15	Vinyard Untrained	200	7068
16	Vinyard Vertical Trellis	200	1607
	total	3200	50929

of BN are utilized, supplemented by three additional layers of GCN and BN. The algorithm employs the Adam optimizer and a softmax cross entropy with logits loss function.

The four benchmark datasets with the training and testing samples for the classification experiments are described as follows.

1) Indian Pines Dataset: The Indian Pines dataset, acquired by the AVIRIS optical sensor over Northwestern Indiana, contains 145 × 145 pixels across 224 spectral bands. After removing 20 noisy bands caused by water absorption, 200 usable bands remain. The dataset includes 16 classes, such as crops, vegetation, roads, and housing, detailed in Table III. The corresponding training and testing sets are shown in Fig. 3.

2) Pavia Dataset: The Pavia HSI is taken over Pavia, Northern Italy, with a Reflective Optics System Imaging Spectrometer (ROSIS). Each band contains 610 × 340 pixels at a geometric resolution of 1.3 m and a total of 103 bands. This scene consists of nine classes mentioned in Table IV. The training and testing set is illustrated in Fig. 6.

3) Houston University Dataset: The Houston University dataset encompasses a total of 15 classes shown in Table V. This hyperspectral dataset was captured using an ITRES-CASI sensor. It comprises 144 spectral bands ranging from 380 to 1050 nm, with a spatial domain of 349 × 1905 pixels per band and a spatial resolution of 2.5 m. The data were collected over the University of Houston campus. The training and testing set used for this dataset is shown in Fig. 8.

4) Salinas Dataset: This dataset was collected by the 224-band AVIRIS sensor over Salinas Valley, California, with a high spatial resolution of 3.7 mpixels. The images are of size 512 lines ×217 samples. As in the Indian Pines scene, 20 water absorption bands 108–112, 154–167, and 224 were eliminated. The land cover classes include vegetables, bare soils, and vineyard fields, totalling 16 ground-truth classes. The training and testing set is displayed in Table VI and illustrated in Fig. 10.

V. RESULTS AND DISCUSSION

In this research, the three commonly used metrics, namely, overall accuracy (OA), average accuracy (AA), and the Kappa score, are used to assess the performance of the S^{3}CLLE-WGCN model. To evaluate its efficacy, a comparative analysis of the classification performance of the GCN without LLE for different weighted adjacency matrices using four different measures, SAM, SID, WD, and RBF over the Indian Pines dataset, is conducted.

TABLE VII
PERFORMANCE OF GCN USING DIFFERENT WEIGHTED ADJACENCY MATRICES WITHOUT EMBEDDING FOR THE INDIAN PINES DATASET

Class No	SAM	SID	WD	RBF
1	81.21	82.66	**89.38**	86.92
2	83.93	82.40	70.79	**86.48**
3	91.85	84.24	91.85	**94.57**
4	93.96	**95.08**	93.96	94.86
5	99.86	**100.00**	98.28	99.86
6	99.32	99.09	**99.77**	99.54
7	78.76	86.38	**86.93**	86.82
8	90.74	88.30	**91.44**	85.73
9	85.46	**92.73**	**92.73**	62.41
10	**100.00**	**100.00**	**100.00**	**100.00**
11	91.56	**91.96**	91.08	91.48
12	97.58	93.03	**98.18**	95.76
13	**100.00**	95.56	**100.00**	**100.00**
14	**100.00**	**100.00**	**100.00**	**100.00**
15	**90.91**	81.82	63.64	**90.91**
16	**100.00**	**100.00**	**100.00**	**100.00**
OA	89.17	89.57	**90.49**	88.40
AA	**92.82**	92.08	91.68	92.21
K	0.876	0.881	**0.891**	0.867

First, the results of the classification performance of the GCN using these metrics for the adjacency matrix construction are shown in Table VII. Bold numbers in the tables highlight the highest accuracies achieved. It can be observed that the results using spectral metrics surpass those achieved with RBF in terms of OA and the Kappa coefficient (κ), which is the common metric used for the creation of the adjacency matrix for GCN [15], [24]. This finding indicates that using spectral metrics to create the adjacency matrix could improve classification performance for HSIs. The same table also shows the classification performance result using the metric proposed in this work WD, which achieved the highest OA of 90.49 and a Kappa coefficient of 0.891. The WD considers the

TABLE VIII
CLASSIFICATION PERFORMANCE OF VARIOUS GCN METHODS FOR THE PAVIA DATASET

class No.	MIniGCN[56]	SSOGCN[51]	$F^2HNNssc$[23]	Non-local GCN[52]	MiniGCN[24]	AN-GCN[15]	**S^{3}CLLE-WGCN**
1	71.99±11.68	95.00	98.05	86.80	96.35	**97.01**	95.96
2	95.56±2.00	**99.09**	98.73	88.74	89.43	86.81	91.43
3	96.87±1.62	84.58	87.47	70.84	87.01	99.53	**99.94**
4	93.04±1.94	95.92	98.60	98.43	94.26	89.65	**99.65**
5	99.37±0.18	99.77	**100.00**	99.85	99.82	**100.00**	**100.00**
6	69.75±11.67	97.23	99.74	94.37	43.12	99.40	**100.00**
7	96.44±1.70	98.28	98.20	86.24	90.96	**100.00**	**100.00**
8	16.46±10.94	96.26	97.07	96.74	77.42	47.79	**99.81**
9	**100.00±0.00**	99.89	98.63	95.78	87.27	**100.00**	94.55
OA (%)	82.19±1.41	97.08	**98.06**	90.04	79.79	88.36	95.23
AA (%)	82.17±0.82	96.22	97.39	90.87	85.07	91.13	**97.92**
k	0.7607±0.170	0.961	**0.9744**	0.8706	0.7367	0.8453	0.936

TABLE IX
CLASSIFICATION PERFORMANCE OF VARIOUS GCN METHODS FOR SALINAS SCENE

Class No	NON-LOCAL[52]	CAD-GCN[55]	SSOGCN[51]	GR-GCN[56]	DRGCN[54]	S^{3}CLLE-WGCN
1	99.69	99.46±1.54	**100.00**	**100±0.00**	**100.00**	98.56
2	99.21	88.08±4.57	**100.00**	**100±0.00**	**100.00**	**100.00**
3	99.79	95.62±1.89	99.88	99.95±0.06	**100.00**	**100.00**
4	98.29	97.85±2.75	**99.75**	99.66±0.43	99.58	98.49
5	99.28	93.79±3.71	97.26	**99.87±0.10**	98.02	99.84
6	99.80	96.41±1.78	**100.00**	99.95±0.08	99.97	**100.00**
7	99.04	97.95±2.84	**100.00**	99.98±0.05	99.89	**100.00**
8	79.11	**99.81±0.25**	83.12	99.17±0.68	99.46	99.58
9	97.74	**100±0.00**	99.80	99.84±0.15	100.00	99.95
10	95.01	89.34±4.03	97.84	99.56±0.43	97.37	**99.94**
11	94.60	93.09±3.14	99.45	**100±0.00**	**100.00**	99.88
12	**100.00**	93.41±2.54	99.94	99.97±0.05	99.83	**100.00**
13	98.96	99.79±0.29	**100.00**	99.97±0.06	98.38	**100.00**
14	99.41	99.36±0.54	**100.00**	99.93±0.10	98.01	**100.00**
15	84.26	99.20±0.87	91.99	99.69±0.17	99.73	**99.84**
16	98.01	98.91±1.01	99.94	**99.95±0.11**	99.51	99.69
OA	92.48	94.13±0.78	95.05	99.71±0.18	99.48	**99.77**
AA	96.39	96.38±0.35	98.06	**99.84±0.09**	99.36	99.74
k	0.920	0.932±0.008	94.49	0.996±0.0020	99.43	**0.997**

entire spectral distribution of each pixel, making it robust against small variations and noise, thereby allowing for better differentiation between similar pixels and making it useful for classifying different materials in HSIs.

Second, various LLE methods, including the novel S^{3}CLLE proposed in this work, are evaluated utilizing the Indian Pines dataset (see Table X). The evaluation involves the application of the four metrics for adjacency matrix computation outlined in Section III. Notably, we observed slight variations in results across metrics for the same LLE method. For instance, for the SLLE method, metrics such as SAM, SID, and WD demonstrated superior classification performance compared to the commonly used RBF metric for constructing the adjacency matrix. Our findings suggest that the SAM metric and WD are promising choices for constructing adjacency matrices. Further comparison between LLE methods revealed that both SLLE and S^{3}CLLE-GCN yielded optimal classification performance, with our method showcasing remarkable superiority over SLLE. Within the S^{3}CLLE-GCN framework, SAM and WD metrics achieved the best classification results, with WD demonstrating a superior distribution of classification among classes, reaching an AA of 95.49%. Based on these observations, WD is selected for the construction of the adjacency matrix in the remainder of our experiments. Consequently, our final model is referred to as S^{3}CLLE-WGCN, emphasizing the integration of S^{3}CLLE and a WD-based GCN architecture for HSI classification.

A. Experimental Results on Indian Pines

A comparison of classification results obtained for the Indian Pines dataset is presented in Table I. It can be observed that the proposed S^{3}CLLE-WGCN method achieved the best

TABLE X
PERFORMANCE OF DIFFERENT EMBEDDING METHODS USING DIFFERENT WEIGHT ADJACENCY MATRIX CONSTRUCTION METRICS FOR THE INDIAN PINES DATASET

	LLE-GCN				SLLE-GCN				SCLLE-GCN				S^{3}CLLE-GCN			
CLASSES	**SAM**	**SID**	**WD**	**RBF**	**SAM**	**SID**	**WD**	**RBF**	**SAM**	**SID**	**WD**	**RBF**	**SAM**	**SID**	**WD**	**RBF**
1	49.71	45.81	48.63	61.34	99.64	99.42	99.28	**100.00**	61.71	60.55	61.49	61.27	99.78	99.78	99.78	**100.00**
2	79.85	79.97	80.87	70.28	99.36	98.60	99.49	44.77	84.95	83.42	82.91	78.70	99.49	98.98	**99.74**	93.62
3	97.28	96.74	97.28	88.04	99.46	99.46	**99.46**	99.46	88.04	85.33	87.50	82.61	99.46	98.37	98.91	99.46
4	83.67	84.34	83.89	83.22	95.30	94.63	95.53	97.54	89.04	88.81	88.81	89.71	97.09	96.20	**96.87**	99.11
5	95.27	95.27	95.41	92.40	99.86	100.00	**100.00**	99.86	91.39	90.67	91.68	90.67	100.00	100.00	**100.00**	100.00
6	97.72	96.36	98.41	98.41	99.77	100.00	**99.77**	99.77	96.81	97.04	97.04	96.81	99.32	99.32	99.09	99.54
7	80.72	78.87	80.83	83.55	97.06	96.30	**96.19**	98.47	83.66	82.24	66.21	87.25	96.08	95.32	95.53	98.58
8	59.47	59.51	63.03	63.65	99.75	99.59	**99.75**	99.92	66.21	65.92	76.42	63.32	99.67	99.59	99.63	98.76
9	77.66	81.38	78.01	68.97	97.34	95.04	**96.63**	98.23	76.95	77.13	**100.00**	77.48	93.26	89.18	94.50	98.58
10	97.53	98.15	97.53	98.15	**100.00**	**100.00**	**100.00**	100.00	100.00	100.00	78.38	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
11	74.52	75.24	74.68	63.34	99.60	99.68	**99.60**	99.44	79.58	79.50	60.91	75.88	97.91	97.75	97.67	97.35
12	88.48	85.76	86.97	90.00	99.39	90.30	**99.70**	99.09	61.52	60.91	0.00	67.58	98.79	97.58	99.39	99.39
13	**100.00**	97.78	**100.00**	97.78	95.56	93.33	95.56	95.56	97.78	97.78	**100.00**	97.78	**100.00**	**100.00**	**100.00**	**100.00**
14	92.31	89.74	**92.31**	89.74	25.64	64.10	28.21	33.33	**100.00**	**100.00**	**100.00**	97.44	56.41	71.79	66.67	66.67
15	63.64	63.64	63.64	81.82	18.18	54.55	36.36	27.27	63.64	63.64	63.64	81.82	72.73	**100.00**	**100.00**	63.64
16	**100.00**	**100.00**	**100.00**	**100.00**	80.00	80.00	80.00	80.00	**100.00**	**100.00**	**100.00**	**100.00**	80.00	80.00	80.00	60.00
AA(%)	83.61	83.03	83.84	83.17	87.87	91.56	89.09	85.79	83.83	83.31	83.67	84.27	93.12	95.24	**95.49**	92.17
OA(%)	72.87	72.32	73.74	72.82	98.66	98.19	**98.55**	94.62	76.47	75.85	75.97	75.16	98.38	97.97	98.44	98.32
K(%)	69.503	68.902	70.416	69.269	98.455	97.921	**98.336**	93.801	73.341	72.643	72.779	71.925	98.134	97.670	98.206	98.077

classification results in terms of AA and the Kappa coefficient (K). Additionally, it exhibited good per-class performance, maintaining a classification accuracy above 97% for 12 out of 16 classes. Fig. 5 shows the Indian Pines HSI whose ground truth is shown in Fig. 4(a), and the classification map is shown in Fig. 4(b) with minimal misclassification between classes. Notably, the S^{3}CLLE-WGCN method performed better at class boundaries, as seen in the classification map enclosed by the yellow box corresponding to class 1 (Corn Notill) and class 8 (Soybean Mintill) in Fig. 4(b). Among the comparison methods, it can be observed that the proposed method achieved the best discrimination between these classes, with a classification accuracy of 98.77% for class 1 and 99.63% for class 8. From this result, we can infer the effect of spatial coherence in the LLE method, which increases the difference between edge pixel spectra, thereby facilitating more accurate classification.

B. Experimental Results on Pavia University

The results obtained for the Pavia dataset are shown in Table VIII. It can be seen that the proposed model achieved the highest AA compared to other methods. Additionally, the proposed method demonstrated superior classification accuracy for classes 2–7, with three of these classes achieving a classification rate of 100%, resulting in the highest per-class classification performance compared to other models. Fig. 7 illustrates the classification map [Fig. 7(a)], which closely matches the ground truth [Fig. 7(a)].

One of the classes with the most misclassification in methods [23], [24], [51], [52] is class 3 (Gravel), and in the methods [15], [24], [56], class 8 (Bricks). However, the S^{3}CLLE-WGCN method achieved the best classification results with accuracies of 99.94% and 99.81% for these classes, demonstrating the effectiveness of the proposed method.

C. Experimental Results on University of Houston

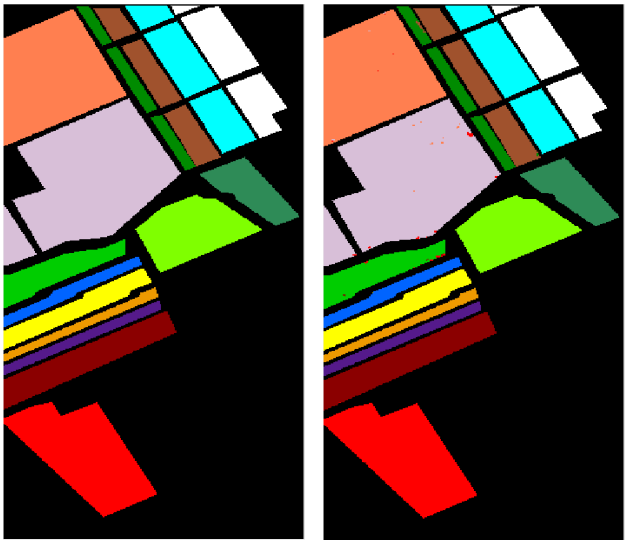
The results for the Houston University dataset are presented in Table II. The proposed method achieved superior classification performance with an OA of 99.28%, an AA of 99.35%, and a Kappa coefficient of 0.992, outperforming other methods in 11 out of the 15 classes. These results underscore the efficiency of the S^{3}CLLE-WGCN method.

In comparison to [15], which utilized the same adaptive method for constructing the adjacency matrix, the feature extraction using the proposed S^{3}CLLE-WGCN method significantly improved classification results for specific classes, such as classes 9 and 13, by over 8%. The classification map is shown in Fig. 9(b). Misclassifications are minimal, with each class achieving over 97% accuracy. Notably, classes 1, 2, 5, 6, 9, and 14 achieved perfect classification.

One of the classes with higher misclassification rates is class 8 (Commercial), a trend also noted in [53] and [57]. Despite this, the S^{3}CLLE-WGCN method achieved a classification accuracy of 99.53% for class 8, the highest among the compared models.

D. Experimental Results on Salinas

The results for the Salinas dataset are presented in Table IX. The S^{3}CLLE-WGCN method achieved the highest OA of 99.77% and the highest Kappa coefficient of 0.997. Furthermore, it achieved perfect classification in classes 2, 3, 6, 7, 10, and 12–15, demonstrating that the S^{3}CLLE-WGCN method surpasses state-of-the-art methods based on GCN. One of the closest comparison methods is GR-GCN, which employs the same training set size and a feature extraction method utilizing a 3DCNN [56]. However, our S^{3}CLLE-WGCN method exceeds their results in both OA and Kappa coefficient, and it also achieves a higher number of perfect classifications per class. These findings highlight the efficacy of the S^{3}CLLE-based feature extraction method proposed in this study, combined with the creation of the adjacency matrix using WD. The Salinas had the challenge of being a large dataset, but the S^{3}CLLE was able to capture the important characteristics of this dataset and represent it in a smaller dimension. Fig. 10 illustrates the classification map [Fig. 11(b)] and the ground truth [Fig. 11(a)] for the Salinas dataset.



(a) (b)

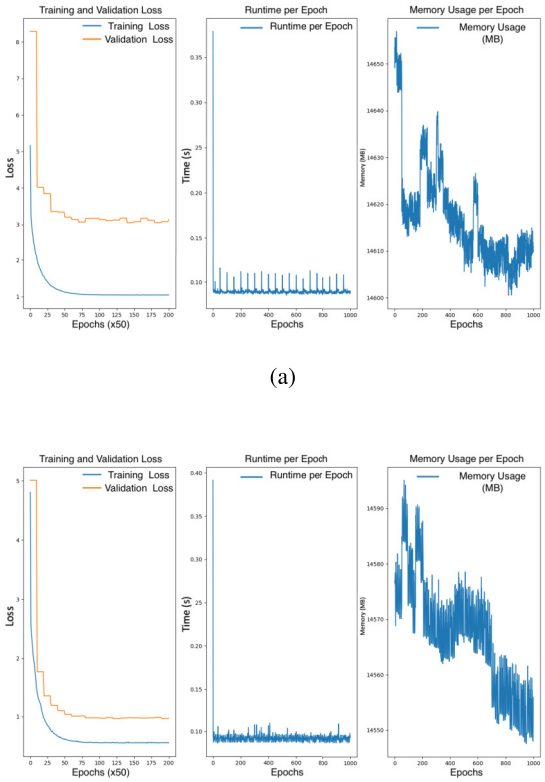
Fig. 11. Classification map for Salinas Scene dataset. (a) Ground-truth map. (b) S^{3}CLLE-WGCN.

The minimal misclassification observed is consistent with the results shown in Table IX.

VI. Analysis

A. Computational Efficiency

The computational efficiency of the S^{3}CLLE-WGCN is analyzed in terms of training and validation loss curves, execution time, and memory usage. The loss curves, time per epoch, and memory used for GCN without any embedding are shown in Fig. 12(a) and for the S^{3}CLLE-WGCN method in Fig. 12(b). It can be observed that for both methods, the training loss and the validation loss initially decrease rapidly, but the loss in Fig. 12(a) for the S^{3}CLLE-WGCN method stabilizes around 1 after some epochs, and for GCN without embedding, the loss stabilizes at 3. This means that the S^{3}CLLE-WGCN method achieves a lower final validation loss, indicating a better generalization capacity, suggesting that it performs better in terms of accuracy. The proposed method shows efficiency in execution time due to the more compact representation by means of embeddings, as can be seen in the center plots in Fig. 12(a) and (b). The S^{3}CLLE-WGCN method uses less memory initially and maintains a lower memory usage throughout compared to the GCN method, as can be seen in the rightmost plot in Fig. 12(a) and (b). With these results, it can be concluded that the S^{3}CLLE-WGCN method shows better performance in terms of accuracy, runtime efficiency, and memory consumption. The S^{3}CLLE reduces the dimensionality of the dataset, making large datasets manageable. The adjacency matrix for all datasets, including Houston, was constructed by applying an adaptive neighborhood selection that limits the number of neighbors based on a dispersion threshold, ensuring that only the most relevant spatial and spectral relationships are preserved. This significantly reduces the size of the adjacency matrix, making it more manageable for large datasets, and ensures that the method remains scalable, even for large



(a)

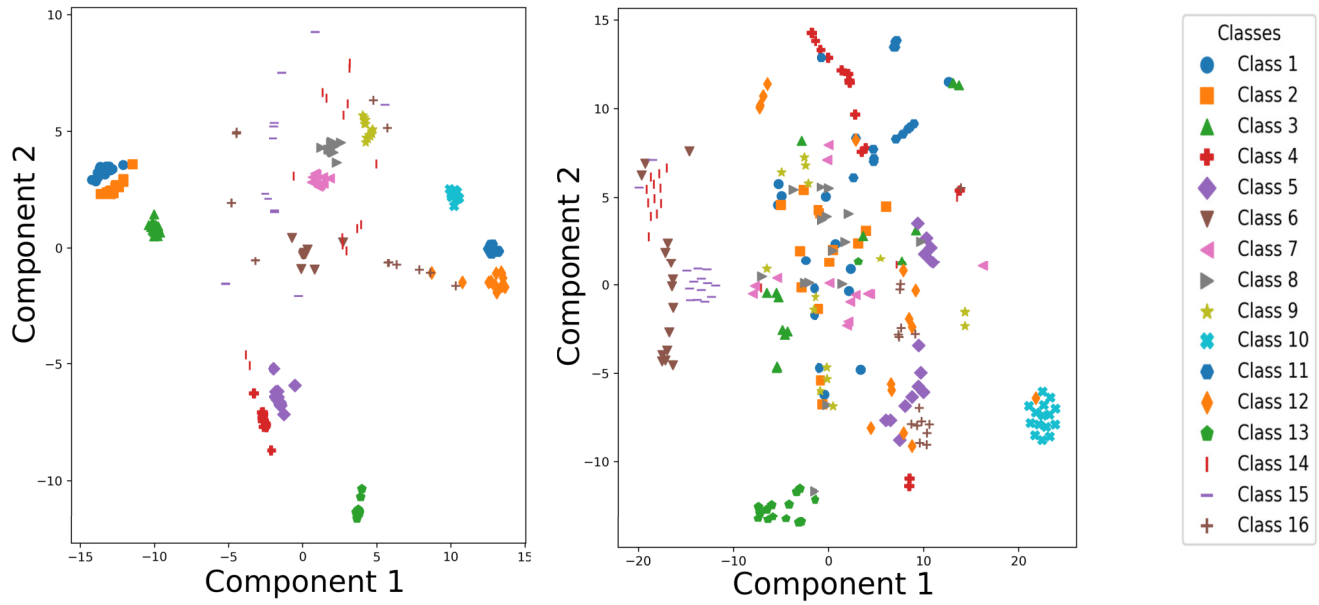
(b)

Fig. 12. (a) Loss curve, timing, and memory consumption using GCN for Indian Pines. (b) Loss curve, timing, and memory consumption for S^{3}CLLE-WGCN for Indian Pines.

datasets like Houston, by focusing on the most relevant neighbors. Additionally, we used efficient matrix storage formats, such as sparse matrices, and employed batch processing during graph construction, ensuring that the method remains scalable and feasible for real-time applications. Compared with other graph-based methods, such as standard GCN or RBF-based approaches, our method is more computationally efficient.

B. S^{3}CLLE Analysis

DR is a critical step in simplifying the high dimensionality of HSI datasets while preserving their intrinsic structure. Existing methods face several limitations, such as the inability to effectively incorporate spatial and spectral information for distinguishing complex land cover classes [58]. Furthermore, many approaches depend heavily on labeled samples, making them impractical due to the high cost of acquiring ground truth in remote sensing applications [59]. Additionally, traditional techniques often need help to preserve local geometric structures during DR, resulting in the loss of essential topological information [60]. The S^{3}CLLE framework directly addresses these challenges by integrating spatial, spectral, and supervised coherence into the embedding process, thereby improving the accuracy and robustness of HSI classification. To evaluate the effectiveness of DR, we employed the silhouette score [61], a metric that quantifies clustering performance. For a single



(a) (b) (c)

Fig. 13. Comparative 2-D visualization of (a) S^{3}CLLE, (b) LLE, and (c) ground truth on the Indian Pines dataset using 15 randomly selected samples per class. Each subfigure includes a 2-D projection of the high-dimensional data.

data point i, the silhouette score is defined as

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{29}$$

where $a(i)$ represents the average distance between i and all other points within the same cluster (coherence), and $b(i)$ is the minimum average distance between i and points in the nearest neighboring cluster (separation). The score ranges from -1 to 1, where higher values indicate well-defined and well-separated clusters.

Fig. 13 provides a comparative visualization of the reduced 2-D spaces generated by S^{3}CLLE and LLE on the Indian Pines dataset. S^{3}CLLE produces more distinct and compact clusters than LLE, underscoring its ability to preserve the dataset's class separability.

The overall clustering performance of DR methods is reflected in their average silhouette scores. S^{3}CLLE achieves a score of **0.597**, indicating well-clustered data points with clear between-class separability and effective preservation of the original dataset's class structure. In contrast, LLE yields a score of **0.003**, highlighting significant overlap between clusters and its inability to provide meaningful discrimination between-class boundaries in the reduced-dimensional representation.

Table X presents a detailed comparison of the classification performance of different DR techniques LLE, SLLE, SCLLE, and S^{3}CLLE, when paired with GCNs on the Indian Pines dataset. The table highlights the classification accuracy using various weight adjacency matrix metrics, including SAM, SID, WD, and RBF, across all 16 classes. The S^{3}CLLE method demonstrates superior performance across the majority of metrics and classes. Notably, it achieves the highest AA of **95.49%**, outperforming the other methods, including SCLLE (**83.83%**) and SLLE (**87.87%**). This improvement can be attributed to the integration of spatial, spectral, and supervised coherence, which enables S^{3}CLLE to preserve both local and global structures essential for HSI classification. Additionally, S^{3}CLLE attains an OA of **98.55%**, closely matching the highest OA of **98.66%** achieved by SLLE while consistently delivering robust results across different adjacency metrics.

The *Kappa coefficient (K)* further validates the effectiveness of S^{3}CLLE, achieving a value of **98.206%**, which indicates a strong agreement between predicted and true labels. By contrast, methods such as LLE show significantly lower performance across all metrics, with an OA of **73.74%** and an AA of **83.84%**, emphasizing the limitations of traditional embedding techniques. These results demonstrate S^{3}CLLE's ability to generate separable, informative embeddings, achieving high accuracy across all classes (Table X) and solidifying its role as a state-of-the-art method for HSI classification. Moreover, Tables I, II, VIII, and IX demonstrate that S^{3}CLLE is generalizable to all the HSI benchmark datasets used for validating HSI classification methods, as these remote sensing datasets confirm to optical imaging laws, and cover varied land cover types.

C. WGCN Analysis

In the context of HSI analysis, the WD provides a measure of the similarity between the spectral distributions of pixels, capturing both spatial and spectral relationships. Unlike Euclidean distance, the WD takes into account the entire spectral distribution across bands, making it more robust to variations and noise often found in hyperspectral data.

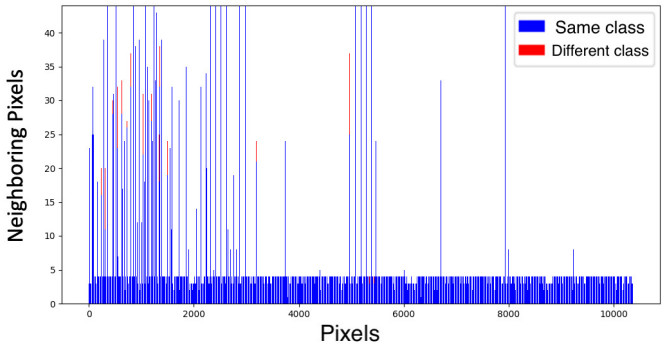


Fig. 14. Adaptive pixel neighborhoods selected in Indian Pines HSI.

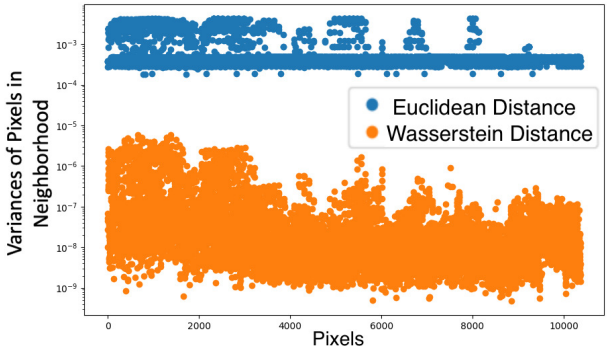


Fig. 15. Variances of weights computed using Euclidean distance and WD for each pixel neighborhood in the adjacency matrix for Indian Pines HSI.

TABLE XI
SMOOTHNESS SCORES FOR ADJACENCY MATRICES BASED ON DIFFERENT METRICS

Metrics	Smoothness score
RBF	15979.07
SAM	1845.26
SID	370.89
WD	345.23

TABLE XII
PERFORMANCE IMPACT OF S^3CLLE

Component	Overall Accuracy (%)
With S^3CLLE	95.49
Without S^3CLLE	90.49

To evaluate the effectiveness of the WD in constructing an adjacency matrix that perfectly represents the spatial–spectral relationships of the underlying HSI data, a smoothness measure (30) is applied to different adjacency matrices computed based on various metrics as shown in Table XI

$$\text{Smoothness score} = \sum_{ij} W_{ij} \left(y_i - y_j\right)^2 \tag{30}$$

where $W_{i,j}$ represents the edge weight between nodes y_i and y_j. This equation captures the degree to which neighboring nodes in a graph (nodes linked by an edge) have similar weights. When nodes have comparable weight values, the smoothness score is low, reflecting consistency in labels across the graph. Conversely, nodes with differing weight values yield a higher smoothness score, indicating dissimilarity [62], [63]. In this work, an adaptive adjacency matrix tailored to HSIs is constructed, where pixels are connected to neighboring pixels that majorly belong to the same class (Fig. 14). As a result, neighboring pixels have similar weights, leading to a low smoothness score that reflects consistency within class boundaries. This configuration suggests that connected pixels are more likely to share the same label, thus effectively capturing class boundaries within the adjacency matrix.

Table XI presents comparative smoothness scores between WD and the other metrics employed, with particular attention to RBF, which is based on Euclidean distance. The results show that WD yields a lower smoothness score, indicating a higher degree of class boundary preservation. This finding supports WD's superior performance in identifying subtle class distinctions in HSIs. The WD is particularly effective in quantifying the shift in spectral distributions between pixels, as WD measures horizontal displacement rather than vertical displacement as measured by the Euclidean distance [64]. Unlike Euclidean distance, which treats each spectral band independently, WD captures the minimal "effort" needed to transform one pixel's spectral distribution into another. This capacity is valuable when two pixels, though representing slightly different materials, exhibit only minor differences in their spectral signatures. WD thus preserves the inherent correlation between spectral bands, offering a more natural metric for such comparisons. In addition, the variance is computed within the neighborhood of each pixel. WD consistently showed lower variance compared to Euclidean distance (Fig. 15), further reinforcing its effectiveness in maintaining similarity within neighborhoods and underscoring its utility in effectively capturing the spatial and spectral relationships in HSI.

TABLE XIII
COMPARISON OF S^3CLLE WITH OTHER DR METHODS USING WGCN

Method	Average Accuracy (%)
S^3CLLE	95.49
LLE	83.84
SLLE	89.09
S^2CLLE	83.67

D. Model Sensitivity and Future Potential

One limitation is the sensitivity of the model to hyperparameters such as neighborhood size (k) for S^3CLLE embedding, which affects classification performance and requires careful tuning for each dataset. However, we have experimentally seen that it works well for $k = 8$. Since the model is supervised, it requires labeled data, which is often limited in some practical applications. To address these considerations, our future work aims to improve the robustness of the model by exploring adaptive strategies for hyperparameter selection for S^3CLLE, thus reducing the reliance on manual tuning. In the case of WGCN, as shown in Fig. 14, the adaptive neighborhood method for adjacency matrix construction selects pixels from

TABLE XIV
PERFORMANCE OF DIFFERENT ADJACENCY MATRIX CONSTRUCTION METHODS

Method	Average Accuracy (%)
Wasserstein Distance (WD)	95.49
Radial Basis Function (RBF)	92.17
Spectral Angle Mapper (SAM)	93.12
Spectral Information Divergence (SID)	95.24

TABLE XV
EFFECT OF VARYING NEIGHBORHOOD SIZES

Neighborhood Size	Average Accuracy (%)
4	93.52
8	95.49
10	91.47
20	93.82

the same class, implying that our method can be extended to unlabeled data, which will be pursued in future work. To address the challenge of minimal spectral variability in pixel classification, we explored various spectral distance metrics and found that the WD effectively distinguishes spectra of similar but distinct classes.

VII. Ablation Study

A. Impact of WGCN

To evaluate the performance improvements introduced by the S^{3}CLLE-WGCN model, a standard GCN was initially implemented on the Indian Pines dataset. The new S^{3}CLLE-WGCN model was then implemented leveraging the WD for computing the weights of the adjacency matrix. The results, summarized in Table VII, highlight the superior performance achieved by employing WD for computing the adjacency matrix weights. Also, a significant improvement in classification accuracy is observed when comparing the results of S^{3}CLLE-WGCN (Table X) with the baseline GCN model. The OA improved from 90.49% to 95.49%, demonstrating the efficacy of the S^{3}CLLE-WGCN method.

B. Impact of S^{3}CLLE

Furthermore, we compared S^{3}CLLE with other DR techniques such as LLE, SLLE, and S^{2}CLLE. The comparison, detailed in Table XIII, demonstrates that S^{3}CLLE consistently outperforms these methods across various metrics when using the WGCN classification model, emphasizing its effectiveness in capturing spatial spectral features and improving classification performance.

As illustrated in Table XIII, S^{3}CLLE achieves higher AA compared to LLE, SLLE, and S^{2}CLLE when using the WGCN classification model, further validating its superiority and critical role in our model.

C. Impact of Different Adjacency Matrix Construction Methods

The impact of various methods for constructing the adjacency matrix was also analyzed. Specifically, the WD method was compared with the RBF, SAM, and SID methods. The results, presented in Table XIV, reveal that the WD method achieves superior performance in terms of accuracy.

D. Effect of Varying S^{3}CLLE Neighborhood Sizes

To identify the optimal neighborhood size for the S^{3}CLLE embedding method, various neighborhood sizes were evaluated, and the corresponding classification performance was analyzed. The results, summarized in Table XV, indicate that a neighborhood size of 8 achieves the highest performance.

VIII. Conclusion

A novel S^{3}CLLE integrated with a WGCN architecture is presented and shown to produce exceptional performance for HSI classification. The S^{3}CLLE-WGCN method combines the strengths of LLE for spatial–spectral embedding with the power of WGCN for effective feature extraction and classification in non-Euclidean spaces. By incorporating spatial and spectral coherence and WD, the S^{3}CLLE-GCN model captures both spectral information and spatial information, leading to superior classification accuracy compared to existing techniques. The experimental results on benchmark datasets such as Indian Pines, Pavia, Houston, and Salinas demonstrate the effectiveness of the S^{3}CLLE-GCN model, achieving the highest average accuracies reported to date.

Our model can be applied to more diverse and challenging HSI datasets. We also aim to investigate the potential of our methodology in real-world applications, such as agricultural monitoring, urban planning, and environmental assessment, where accurate HSI classification is crucial. The promising results of this study underscore the potential of S^{3}CLLE-WGCN in advancing the field of HSI analysis, providing a robust framework for both academic research and practical implementations.

Acknowledgment

The authors would like to thank the ExpandAI project at UPRM and the NSF Institute for Artificial Intelligence and Fundamental Interactions (IAIFI) at MIT for their support. Opinions, findings, conclusions, or recommendations expressed in this material are those of their and do not necessarily reflect the views of the sponsors.

References

[1] T. A. Moughal, "Hyperspectral image classification using support vector machine," *J. Phys., Conf. Ser.*, vol. 439, Jun. 2013, Art. no. 012042.
[2] O. Okwuashi and C. E. Ndehedehe, "Deep support vector machine for hyperspectral image classification," *Pattern Recognit.*, vol. 103, Jul. 2020, Art. no. 107298.
[3] S. D. Manzanarez and V. Manian, "Supervised spatially coherent nonlinear dimensionality reduction for hyperspectral image classification," in *Proc. 11th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, Mar. 2021, pp. 1–5.
[4] R. Gogineni and A. Chaturvedi, "Hyperspectral image classification," in *Processing and Analysis of Hyperspectral Data*, J. Chen, Y. Song, and H. Li, Eds., Rijeka, Croatia: IntechOpen, 2019, ch. 2. [Online]. Available: https://doi.org/10.5772/intechopen.88925
[5] L. Gao, S. Xiao, C. Hu, and Y. Yan, "Hyperspectral image classification based on fusion of convolutional neural network and graph network," *Appl. Sci.*, vol. 13, no. 12, p. 7143, Jun. 2023.
[6] C. Shi, J. Sun, and L. Wang, "Hyperspectral image classification based on spectral multiscale convolutional neural network," *Remote Sens.*, vol. 14, no. 8, p. 1951, Apr. 2022. [Online]. Available: https://www.mdpi.com/2072-4292/14/8/1951

[7] X. Cao, J. Yao, Z. Xu, and D. Meng, "Hyperspectral image classification with convolutional neural network and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 7, pp. 4604–4616, Jul. 2020.

[8] E. Nasr-Esfahani et al., "Dense pooling layers in fully convolutional network for skin lesion segmentation," 2017, *arXiv:1712.10207*.

[9] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, vol. 9351. Cham, Switzerland: Springer, 2015, pp. 234–241.

[10] P. F. Christ et al., "Automatic liver and lesion segmentation in CT using cascaded fully convolutional neural networks and 3D conditional random fields," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, 2016, pp. 415–423.

[11] L. Jing, Y. Chen, and Y. Tian, "Coarse-to-fine semantic segmentation from image-level labels," *IEEE Trans. Image Process.*, vol. 29, pp. 225–236, 2020.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 630–645.

[13] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," 2019, *arXiv:1905.06133*.

[14] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[15] J. Orozco, V. Manian, E. Alfaro, H. Walia, and B. K. Dhatt, "Graph convolutional network using adaptive neighborhood Laplacian matrix for hyperspectral images with application to rice seed image classification," *Sensors*, vol. 23, no. 7, p. 3515, Mar. 2023.

[16] Y. Lu, Y. Chen, D. Zhao, and J. Chen, "Graph-FCN for image semantic segmentation," 2020, *arXiv:2001.00335*.

[17] F. Ma, F. Gao, J. Sun, H. Zhou, and A. Hussain, "Attention graph convolution network for image segmentation in big SAR imagery data," *Remote Sens.*, vol. 11, no. 21, p. 2586, Nov. 2019. [Online]. Available: https://www.mdpi.com/2072-4292/11/21/2586

[18] H. Li, W. Xiong, Y. Cui, and Z. Xiong, "Explainable classification of remote sensing ship images based on graph network," *Chin. J. Inf. Fusion*, vol. 1, no. 2, pp. 126–133, Sep. 2024.

[19] X. Jin et al., "YOLOv7-bw: A dense small object efficient detector based on remote sensing image," *IECE Trans. Intell. Systematics*, vol. 1, no. 1, pp. 30–39, May 2024.

[20] R. Hanachi, A. Sellami, I. R. Farah, and M. Dalla Mura, "Multi-view graph representation learning for hyperspectral image classification with spectral–spatial graph neural networks," *Neural Comput. Appl.*, vol. 36, no. 7, pp. 3737–3759, Mar. 2024.

[21] C. Ding et al., "Integrating prototype learning with graph convolution network for effective active hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504816, doi: 10.1109/TGRS.2024.3352112.

[22] W. Cai et al., "Graph-structured convolution-guided continuous context threshold-aware networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5527618, doi: 10.1109/TGRS.2023.3323530.

[23] Z. Ma, Z. Jiang, and H. Zhang, "Hyperspectral image classification using feature fusion hypergraph convolution neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517314.

[24] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[25] S. Jia, S. Jiang, S. Zhang, M. Xu, and X. Jia, "Graph-in-graph convolutional network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 1157–1171, Jan. 2024.

[26] M. R. Islam, B. Ahmed, M. A. Hossain, and M. P. Uddin, "Mutual information-driven feature reduction for hyperspectral image classification," *Sensors*, vol. 23, no. 2, p. 657, Jan. 2023.

[27] D. Ruiz Hidalgo, B. Bacca Cortés, and E. Caicedo Bravo, "Dimensionality reduction of hyperspectral images of vegetation and crops based on self-organized maps," *Inf. Process. Agricult.*, vol. 8, no. 2, pp. 310–327, Jun. 2021.

[28] A. Mohan, G. Sapiro, and E. Bosch, "Spatially coherent nonlinear dimensionality reduction and segmentation of hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 4, no. 2, pp. 206–210, Apr. 2007.

[29] B. Rasti et al., "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag. (replaces Newsletter)*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[30] B. Ghojogh et al., "Feature selection and feature extraction in pattern analysis: A literature review," 2019, *arXiv:1905.02845*.

[31] C.-I. Chang and Q. Du, "Interference and noise-adjusted principal components analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 37, no. 5, pp. 2387–2396, Sep. 1999.

[32] L. Zhang and F. Luo, "Review on graph learning for dimensionality reduction of hyperspectral image," *Geo-Spatial Inf. Sci.*, vol. 23, no. 1, pp. 98–106, Jan. 2020.

[33] L. K. Saul and S. T. Roweis. (2001). *An Introduction to Locally Linear Embedding*. [Online]. Available: http://www.cs.toronto.edu/roweis/lle/publications.html

[34] D. d. Ridder, O. Kouropteva, O. Okun, M. Pietikäinen, and R. P. W. Duin, "Supervised locally linear embedding," in *Proc. Int. Conf. Artif. Neural Netw.*, 2003, pp. 333–341.

[35] J. Chen and Y. Liu, "Locally linear embedding: A survey," *Artif. Intell. Rev.*, vol. 36, no. 1, pp. 29–48, Jun. 2011.

[36] S. T. Roweis and L. K. Saul, "Nonlinear dimensionality reduction by locally linear embedding," *Science*, vol. 290, no. 5500, pp. 2323–2326, Dec. 2000.

[37] C. Varini, A. Degenhard, and T. W. Nattkemper, "ISOLLE: LLE with geodesic distance," *Neurocomputing*, vol. 69, nos. 13–15, pp. 1768–1771, Aug. 2006.

[38] Y. Pan, S. S. Ge, and A. Al Mamun, "Weighted locally linear embedding for dimension reduction," *Pattern Recognit.*, vol. 42, no. 5, pp. 798–811, May 2009.

[39] H. Wang, J. Zheng, Z. Yao, and L. Li, "Improved locally linear embedding through new distance computing," in *Proc. 3rd Int. Symp. Neural Netw.*, Chengdu, China. Cham, Switzerland: Springer, 2006, pp. 1326–1333.

[40] S.-Q. Zhang, "Enhanced supervised locally linear embedding," *Pattern Recognit. Lett.*, vol. 30, no. 13, pp. 1208–1218, Oct. 2009.

[41] T. Boucher et al., "Manifold preprocessing for laser-induced breakdown spectroscopy under Mars conditions," *J. Chemometrics*, vol. 29, no. 9, pp. 484–491, 2015.

[42] F. F. Shahraki and S. Prasad, "Graph convolutional neural networks for hyperspectral data classification," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, Nov. 2018, pp. 968–972.

[43] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[44] Z. Gong et al., "Superpixel spectral–spatial feature fusion graph convolution network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5536216, doi: 10.1109/TGRS.2021.3075223.

[45] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, Mar. 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1063520310000552

[46] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[47] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203*.

[48] A. Ramdas, N. Trillos, and M. Cuturi, "On Wasserstein two-sample testing and related families of nonparametric tests," *Entropy*, vol. 19, no. 2, p. 47, Jan. 2017.

[49] F. A. Kruse et al., "The spectral image processing system (SIPS)—Interactive visualization and analysis of imaging spectrometer data," *Remote Sens. Environ.*, vol. 44, nos. 2–3, pp. 145–163, May 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/003442579390013N

[50] C.-I. Chang, "An information-theoretic approach to spectral variability, similarity, and discrimination for hyperspectral image analysis," *IEEE Trans. Inf. Theory*, vol. 46, no. 5, pp. 1927–1932, May 2000.

[51] M. Zhang, H. Luo, W. Song, H. Mei, and C. Su, "Spectral–spatial offset graph convolutional networks for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 21, p. 4342, Oct. 2021. [Online]. Available: https://www.mdpi.com/2072-4292/13/21/4342

[52] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257, Dec. 2020.

[53] S. Wan et al., "Dual interactive graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5510214, doi: 10.1109/TGRS.2023.3240721.
[54] R. Chen, G. Li, and C. Dai, "DRGCN: Dual residual graph convolutional network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[55] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 597–612, Jan. 2021.
[56] C. Zhang, J. Wang, and K. Yao, "Global random graph convolution network for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 12, p. 2285, Jun. 2021. [Online]. Available: https://www.mdpi.com/2072-4292/13/12/2285
[57] W. Yu, S. Wan, G. Li, J. Yang, and C. Gong, "Hyperspectral image classification with contrastive graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023.
[58] W. Zhao and S. Du, "Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 8, pp. 4544–4554, Aug. 2016.
[59] G. Camps-Valls, T. V. B. Marsheva, and D. Zhou, "Semi-supervised graph-based hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 10, pp. 3044–3054, Oct. 2007.
[60] Y. Fang et al., "Dimensionality reduction of hyperspectral images based on robust spatial information using locally linear embedding," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 10, pp. 1712–1716, Oct. 2014.
[61] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0377042787901257
[62] M. Daković, L. Stanković, and E. Sejdić, "Local smoothness of graph signals," *Math. Problems Eng.*, vol. 2019, no. 1, Jan. 2019, Art. no. 3208569.
[63] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, and J. Tang, "Graph structure learning for robust graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 66–74.
[64] S. Kolouri, S. R. Park, M. Thorpe, D. Slepcev, and G. K. Rohde, "Optimal mass transport: Signal processing and machine-learning applications," *IEEE Signal Process. Mag.*, vol. 34, no. 4, pp. 43–59, Jul. 2017.



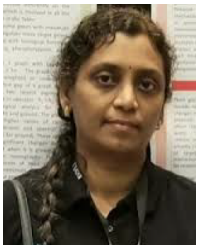
Jairo Orozco (Member, IEEE) received the bachelor's degree in physics from the Universidad del Atlántico, Barranquilla, Colombia, in 2016, and the master's degree in physics from the University of Puerto Rico at Mayagüez (UPRM), Mayagüez, PR, USA, in 2019, where he is currently pursuing the Ph.D. degree in electrical engineering.

He is currently a Researcher with the Artificial Intelligence Imaging Group, UPRM. He has five refereed journal publications in machine learning. His research interests include signal processing, pattern recognition, machine learning, and condensed matter physics.



Sergio Manzanarez (Member, IEEE) received the bachelor's degree in electrical engineering from the Universidad Nacional Autónoma de Honduras, Tegucigalpa, Honduras, in 2013, and the master's degree in applied mathematics from the University of Puerto Rico at Mayagüez (UPRM), Mayagüez, PR, USA, in 2018, where he is currently pursuing the Ph.D. degree in electrical engineering.

He has been the Coordinator and an Instructor of Electrical and Electronics Engineering Laboratories, UPRM. He is also a Researcher at the Artificial Intelligence Imaging Group, UPRM. In addition, he has taught Precalculus, Calculus, Linear Algebra, and Circuit Analysis. He is also a Trainer of the Mathematics Olympiad in Puerto Rico (OMPR). He has three journal publications. His research interests include signal processing, pattern recognition, machine learning, numerical analysis, and mathematics.



Vidya Manian (Senior Member, IEEE) received the B.E. degree in electrical engineering from the A. C. College of Engineering and Technology, Karaikudi, India, in 1990, and the M.S. degree in electrical engineering and the Ph.D. degree in computing, information science and engineering from the University of Puerto Rico at Mayagüez, Mayagüez, PR, USA, in 1995 and 2004, respectively.

She was a Visiting Scholar at West Virginia University, Morgantown, WV, USA, in 2005. Since 2006, she has been the Electrical and Computer Engineering and Bioengineering Faculty, UPRM. She is currently the Director of the Artificial Intelligence Imaging Group Laboratory, UPRM. She has about 25 refereed journal publications and 50 conference publications. Her research interests include predictive artificial intelligence modeling, machine learning, hyperspectral imaging, and bioinformatics.